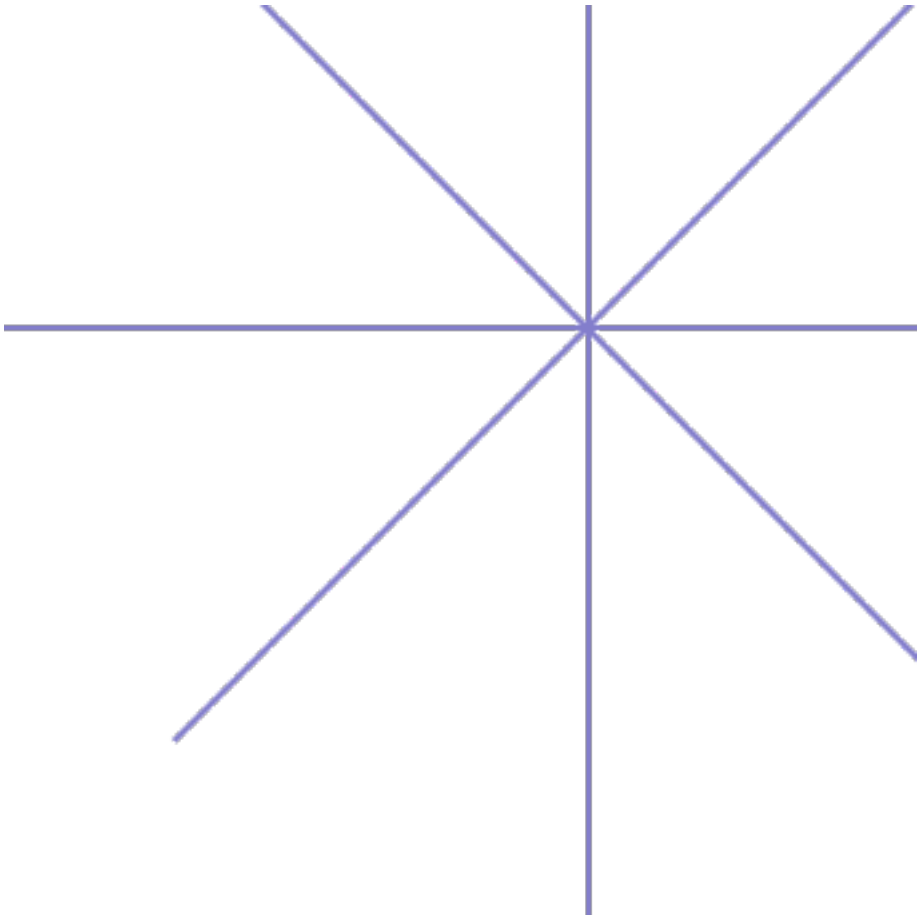


OPIS

Final Report

JEDEJ

Jennifer Butcher	*Client liaison, frontend*
Derek Chai	*UI/UX, frontend*
Elle Fleming	*Backend*
Ethan Gee	*DevOps, rendering*
Jianing Li	*Rendering*

Version 1.0

2. Executive summary

ASTRO 100 — Planets, Stars, and Galaxies is an introductory course into astronomy offered at the University of Auckland / Waipapa Taumata Rau. We developed OPIS (*Orbital Planetary Interactive Simulator*) for our client Dr. Larissa Markwardt[1] and the ASTRO 100 teaching staff in order to provide an application capable of replacing the existing tools used in the ASTRO 100 labs, which are externally managed and unreliable.

OPIS is a cross-platform Electron desktop application built using Tailwind (UI) and Three.js (rendering) on the frontend, which communicates with our REBOUND-based FastAPI backend for accurate and real-time astronomical calculations.

To begin our investigation, we developed our aims and objectives for OPIS, by exploring its target audience and the value of interactive simulations to produce an initial specification. We investigated existing tools to understand where these applications are lacking, and where OPIS could provide value.

From all the testing sessions we conducted, we found OPIS met the majority of our project goals and user requirements. Users can use OPIS to view and interact with the three included planetary systems, which are accurately simulated in 3D. They can play the simulations at different speeds, step through time, and view the planetary systems from different camera angles. However, the most notable area which still needs work is accessibility (keyboard navigation and screen-size responsiveness).

There remain recommendations for areas of future improvement for OPIS that were not able to be developed. The support for arbitrary systems via an API, object following, and proportional object sizes are further features that would enhance students' learning when using OPIS. Furthermore, the modifiability of OPIS is limited and could be increased by introducing automated testing and more developer-facing documentation.

Meeting all of its specific user requirements and the vast majority of its project goals, OPIS is well-suited to help both ASTRO students and teaching staff for years to come.

[1] https://profiles.auckland.ac.nz/larissa-markwardt

3. Table of contents

1. Title page **1**

2. Executive summary **2**

3. Table of contents **3**

4. Introduction **7**

4.1. Aims and objectives . . . 7

4.2. Target Audience . . . 7

4.3. Initial scope . . . 8

4.4. Approach . . . 8

4.5. Important outcomes . . . 9

5. Background **10**

5.1. Project background . . . 10

5.1.1. The value of interactive simulations . . . 10

5.1.2. The ASTRO 100 laboratory . . . 10

5.2. Existing solutions . . . 10

5.2.1. Limitations to existing solutions . . . 10

5.3. Project value . . . 12

5.4. Methods and tools . . . 12

6. Specification and design **14**

6.1. User requirements . . . 14

6.1.1. Performance . . . 14

6.1.2. Functionality . . . 14

6.1.3. Deployment and packaging . . . 15

6.1.4. Accessibility . . . 15

6.2. Use cases . . . 15

6.2.1. ASTRO 100 student . . . 15

6.2.2. ASTRO 100 tutor . . . 23

6.2.3. Maintainers . . . 24

6.3. Design . . . 29

6.3.1. UI/UX . . . 29

6.3.2. System architecture . . . 33

6.3.3. Backend . . . 34

6.3.4. Renderer . . . 35

6.3.5. State management . . . 37
6.3.6. Event system . . . 38
6.3.7. Distribution and updates . . . 39
7. Implementation **41**
7.1. UI/UX implementation . . . 41
7.1.1. Markup . . . 41
7.1.2. Calendar . . . 45
7.1.3. Styling . . . 47
7.1.4. Interactivity . . . 49
7.1.5. Navigation flashes . . . 52
7.1.6. Help menu . . . 54
7.2. Simulation renderer . . . 57
7.2.1. Rendering framework . . . 57
7.2.2. Rendering logic . . . 57
7.2.3. Objects and orbits . . . 58
7.2.4. Orbit line projections . . . 60
7.2.5. Habitable zone . . . 62
7.2.6. Tapered orbital lines . . . 62
7.2.7. Reference grid . . . 64
7.2.8. Raycasting . . . 66
7.3. State management . . . 66
7.3.1. State files . . . 66
7.3.2. State schemas . . . 67
7.3.3. Fetching and saving state . . . 67
7.3.4. Applying state on page load . . . 68
7.3.5. Resetting state . . . 68
7.4. Event system . . . 69
7.5. Electron . . . 70
7.6. Backend . . . 72
7.6.1. FastAPI and Uvicorn . . . 73
7.6.2. REBOUND and astronomical calculations . . . 75
7.6.3. Maintaining interactivity . . . 77
7.6.4. Data sourced . . . 79
7.7. Distribution and updates . . . 80
7.7.1. CD pipeline . . . 80

7.7.2. Auto-updater . . . 81
8. Results and evaluation **82**
8.1. Client testing 1 (25 September 2026) . . . 82
8.1.1. Test plan . . . 82
8.1.2. Results . . . 82
8.1.3. Actions . . . 83
8.2. Peer testing (25 September 2026) . . . 86
8.2.1. Test plan . . . 86
8.2.2. Results . . . 86
8.2.3. Actions . . . 86
8.3. User testing . . . 86
8.4. Client testing 2 (13 October 2026) . . . 86
8.4.1. Test plan . . . 86
8.4.2. Results . . . 87
8.4.3. Actions . . . 88
8.5. Project goals evaluation . . . 90
8.6. Overall evaluation . . . 93
9. Future work **96**
9.1. Arbitrary planetary systems . . . 96
9.2. Object following . . . 96
9.3. Proportional object sizes . . . 97
9.4. Automated tests . . . 98
9.4.1. Frontend and rendering tests . . . 98
9.4.2. Backend tests . . . 98
9.5. Developer-facing documentation . . . 99
10. Conclusion **100**
10.1. Project aims and objectives . . . 100
10.2. Key findings . . . 100
10.3. Major outcomes . . . 102
11. References **103**
12. Appendices **104**
12.1. Gantt chart . . . 104
12.2. Final build . . . 104
12.3. Demonstration video . . . 105
12.4. Document links . . . 105

12.5. Figures 106
12.6. Tables 107
12.7. Listings 107
12.8. Glossary 108
12.9. Table of signatures 109

4. Introduction

4.1. Aims and objectives

Ethan Gee

OPIS aims to be an intuitive, accurate, and reliable tool that helps ASTRO 100 students work through their lab sheet. We consider our project successful if it meets the following objectives:

- **Course specific**: OPIS should provide all of the functionality necessary to complete the ASTRO 100 lab sheet, without any unrelated features which could otherwise distract students from the lab
- **Intuitive**: Students should be able to find and use the functionality needed for each task on the lab sheet, with at most minimal technical assistance from teaching assistants
- **Scientifically accurate**: The information and simulations that OPIS presents should be accurate
- **Reliable**: OPIS should work consistently during the lab, performing well regardless of how many students are using it at once and should run on any laptop or desktop computer
- **Modifiable**: The ASTRO 100 lab may change in the future, for example to include another exoplanetary system or different asteroids or comets; the ASTRO 100 teaching team should therefore be able to make such modifications without prior familiarity with the codebase, so that OPIS can continue to provide value as the course evolves

4.2. Target Audience

Derek Chai

ASTRO 100 is an introductory astronomy course offered at the University of Auckland / Waipapa Taumata Rau. It is offered in both science and general education streams.

Students enrolled in ASTRO 100 come from a wide range of academic backgrounds within the University. They have varying levels of domain knowledge and proficiency in using technical tools and software.

We are developing OPIS so that students enrolled in ASTRO 100 of all backgrounds can find the software intuitive, easy to use, and useful as an aid in answering questions within the course's labs.

4.3. Initial scope

Elle Fleming

In collaboration with advice from our client, we decided OPIS' scope to be that it must:

- Provide simulations for the Solar System, and exoplanetary systems Kepler-16 and TRAPPIST-1.
- Play and step through time, seeing the speeds of various objects in the systems
- Manipulate 3D space with zooming, rotation and panning
- Compare the exoplanet systems to the Solar System
- Show and hide objects within the system
- Show the habitable zones of the systems
- Be intuitive and accessible for ASTRO 100 students
- Be performant and resilient under conditions experienced within the lab

4.4. Approach

Elle Fleming

To facilitate the creation of OPIS, we employed Agile and managed our tasks visually through our GitHub Project's kanban board. We assigned each atomic piece of work a GitHub issue and delegated each issue to a person.

We followed a continuous flow-development model by picking tasks from the kanban board by priority and completing them. Our team meetings assisted this by serving as regular check-ins.

Team meetings were held every Monday and Friday. All team meetings throughout the duration of the project had complete attendance of all team members. Our team meetings were mostly in-person, with the exception of online meetings during the mid-semester break. These meetings would typically last two hours and were an essential part of our approach. During each meeting we would:

- Evaluate our workload during the last work period to ensure we were distributing work evenly
- Gather our opinions on features in-progress, completed, and future
- Review pending pull requests
- Discuss problems and issues encountered to obtain outside opinions
- Plan for the short term by assigning and discussing GitHub issues
- Discuss our plans for beyond the next work period

- Plan for upcoming client meetings

We would also take extensive meeting minutes during each meeting to document everything we discussed so that we could reference the minutes when necessary and to be prepared for if a team member could not attend in entirety.

We also had six fortnightly check-in meetings with our client which were held every second Tuesday. This were typically digital as per our client's request with the exception of the final meeting held in person in week 11. These meetings facilitated crucial feedback from the client as we had the opportunity to present our progress and ensured we were staying aligned with what our client expected. During these meetings we were able to ask a variety of domain-specific questions and get expert advice from our client, and we were able to pitch the features we came up with to the client to better understand the direction we needed to move in.

Before our weekly Friday team meeting, we would have our tutor check-in meeting. We would discuss the progress we made in the past week and receive feedback on our progress. We also would share any roadblocks which were impeding our ability to progress, and our tutor, where possible, would guide us toward potential approaches to investigate how to tackle any problems.

4.5. Important outcomes

Ethan Gee

We have delivered OPIS, a single cross-platform desktop application built specifically for ASTRO 100 to replace the two external tools previously used in the lab. It provides real-time simulations of the Solar System, Kepler-16, and TRAPPIST-1, with the functionality to assist students in completing the lab sheet. Its interface is designed to be intuitive for students from any background.

The simulations are scientifically grounded, built on an established N-body integrator and data from NASA and peer-reviewed sources. OPIS runs locally and works offline, so it performs reliably no matter how many students are using it at once.

OPIS is also built to evolve with the course, making it easy for the teaching team to introduce new systems over time. We are confident that it is capable of serving the ASTRO 100 teaching team's needs for years to come.

5. Background

5.1. Project background

Jennifer Butcher

5.1.1. The value of interactive simulations

Interactive Solar System simulations have been shown to improve students' conceptual understanding and motivation to learn compared with teaching the same content without simulations [1]. These benefits have been proven to hold regardless of a student's prior knowledge [2]. As such, simulations present a valuable tool for improving student learning outcomes in university courses.

5.1.2. The ASTRO 100 laboratory

One ASTRO 100 lab session uses interactive simulations to explore planetary orbits, both in our Solar System and in a selection of exoplanet systems. Students use these simulations to support them as they work through a lab sheet, and answer questions based on their observations.

Each lab session has around 80 students, split into around 20 groups.

5.2. Existing solutions

The ASTRO 100 lab currently relies on two external websites in the lab: *Orbit Viewer* [3] and *Eyes on Exoplanets* [4]. However, both tools have limitations that affect students and the teaching team's experience.

5.2.1. Limitations to existing solutions

- **Designed for a general audience**. Both websites cater to a general audience, rather than ASTRO 100 specifically. For example, *Eyes on Exoplanets* lets users browse over 6,000 exoplanets and filter by planet type, missions, or observatory, offering far more functionality than students require. This excess functionality can make it overwhelming for students to navigate and find the information they're looking for without guidance from teaching staff.

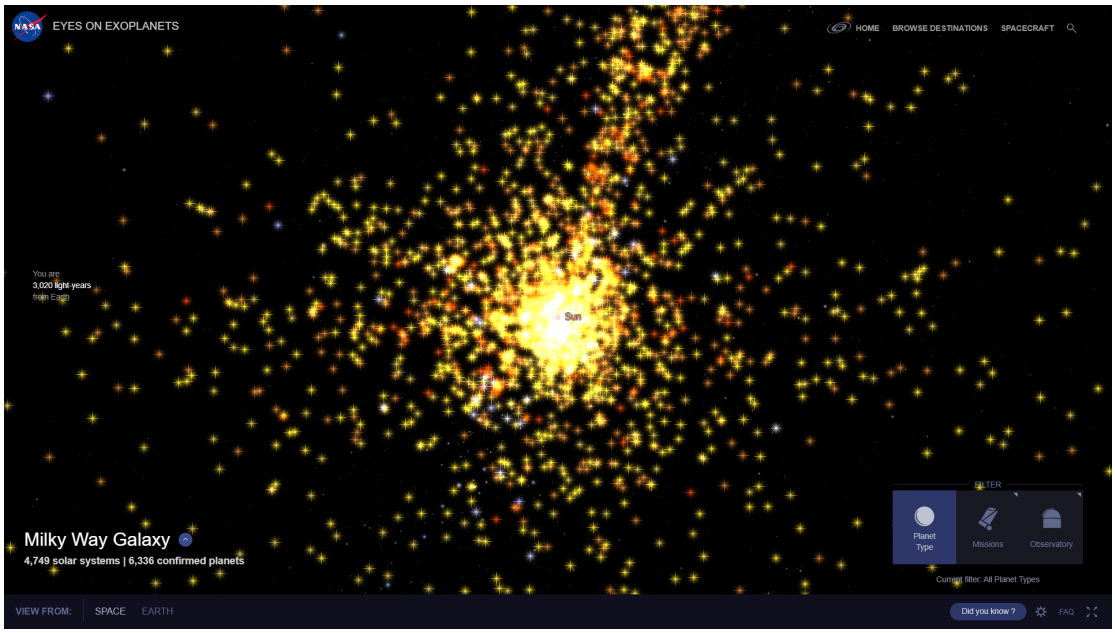


Figure 1

NASA's Eyes on Exoplanets *interface upon first launch.*

- **A fragmented learning experience**. Students must switch between two separate tools, as each has slightly different functionality. This forces them to reorient themselves between two different interfaces, a process which has been shown to impose extra cognitive load unrelated to the material itself, and can even lead to a more negative perception of the learning experience [5].
- **Unreliable under load**. The ASTRO 100 teaching team has reported reliability issues when many students access the websites simultaneously. Separately, the tools tend to slow down or freeze if left running for extended periods.
- **No control for the teaching team**. The teaching team has no way to fix issues with these websites directly, add functionality specific to ASTRO 100, or guarantee that either website will remain available in its current form. If one of the websites changes, the teaching team may not even be aware until students report issues, and the lab sheet can become out of date and require revision, adding an ongoing burden to the teaching team.

5.3. Project value

Jennifer Butcher

Our proposed solution, OPIS, will address these limitations by providing a purpose-built desktop application that combines core functionality of both existing tools with additional features specific to the ASTRO 100 lab.

- **A tailored learning experience**. As OPIS is tailored to the ASTRO 100 lab, its interface will be designed around the lab's requirements, and the functionality students require easy to find. Bringing both simulations into a single application also removes the friction of switching between tools.
- **Reliability and offline use**. OPIS will run locally on the user's computer and work offline, so performance will not be dependent on external servers, or by many students are using the application simultaneously.
- **Ownership for the teaching team**. Because OPIS will be owned and managed directly by the ASTRO 100 teaching team, the team can directly address any issues that may arise, and the application can be updated as the lab content changes.

Together, these changes will give students back the time and attention currently lost to unfamiliar interfaces and technical friction, while giving the teaching team direct ownership and lasting control of the tool going forward.

5.4. Methods and tools

Jennifer Butcher

We will implement OPIS as a desktop application, using modern tools and technologies to create a robust, responsive user experience. We have prioritised technologies that the ASTRO 100 teaching team are already familiar with, such as Python and REBOUND, to ease future maintenance.

To keep our team coordinated and organised, we are also using a suite of tools to manage the project. Table 1 summarises the core tools and technologies used.

Table 1

The tools and technologies utilised throughout the development of OPIS.

Tool	Category	Use
HTML	Frontend	User interface structure
Tailwind CSS	Frontend	User interface styling

Tool	Category	Use
JavaScript	Frontend	User interaction and UI logic handing; communication with backend
Floating UI	Frontend	Positioning library for help tooltips
Three.js	Rendering	3D visualisations of planetary systems rendering
Jinja	Backend	HTML pages and components templating
Python	Backend	Application logic and calculations implementation
FastAPI	Backend	API between the frontend and backend
Uvicorn	Backend	FastAPI web server framework
REBOUND	Backend	N-body integrator for the simulation calculations
Node.js	Backend	Runs the Electron application and build tools
Electron	Platform	Application framework
Figma	Design	UI mockup design prior to implementation
GitHub Projects	Planning	Task organisation; progress tracking using kanban board
GanttPRO	Planning	Project timeline and milestones planning
GitHub	Development	Source code hosting, version control, collaboration, and distribution/release management
GitHub Actions	Development	CI/CD
Typst	Documentation	Project proposal / final report typesetting
Graphviz	Documentation	Final report diagrams
Draw.io	Documentation	Final report diagrams

6. Specification and design

6.1. User requirements

Derek Chai

These user requirements for OPIS were developed in consultation with our client, based off the problems experienced with existing solutions during lab sessions. We iterated on these set of requirements across the course of development, in conjunction with advice and feedback from our client.

6.1.1. Performance

- Load application on lab computers used in lab within 5000 ms from launch
- Load simulations within 1000 ms of user clicking relevant button
- Do not crash when used under normal conditions[2] in lab
- Do not lag noticeably or stutter for longer than 100 ms after any user interaction under normal conditions
- Run simulation rendering consistently at frame rate of at least 30 frames per second on lab computers under normal conditions
- Support up to 80 concurrent users without noticeable performance impact

6.1.2. Functionality

- Observe accurate movement and orbits of objects within the Solar System, Kepler-16, and TRAPPIST-1
- Compare directly orbits of included exoplanetary systems[3] to those of the Solar System
- Overlay each planetary system's habitable zone over its orbits
- Play and pause simulation playback
- Adjust simulation playback speed
- Select an arbitrary date/time and have OPIS show positions of objects at that timestamp
- Rotate and pan around systems in three dimensions
- Select simulated objects to obtain information about their masses, radii, object type, and mean temperature

[2] The conditions under which OPIS is expected to be used in answering the specific questions provided in the lab worksheet

[3] Kepler-16 and TRAPPIST-1.

- Do not contain any unnecessary features—i.e. any features which are not directly involved in answering worksheet questions, or which cannot be used to answer such questions
- Simulate all planets and stars of the Solar System, Kepler-16, and TRAPPIST-1
- Simulate within the Solar System Halley's Comet and comet 3I/ATLAS in addition to all planets and stars
- Show realistic orbits, object positions, and distances to level of accuracy equal to or better than existing tools[4]

6.1.3. Deployment and packaging

- Be installable on lab computers (i.e. on Windows devices)
- Provide all required functionality and user requirements in one single executable

6.1.4. Accessibility

- Be accessible to users with colour vision deficiencies
- Be accessible to users with dyslexia or reading-related difficulties
- Be useable with only the keyboard
- Be responsive to all screen sizes

6.2. Use cases

Derek Chai

Here we describe 3 potential use cases for OPIS:

6.2.1. ASTRO 100 student

ASTRO 100 students interact with OPIS in a 1-hour lab session. In these labs, students work in small groups to answer a provided lab worksheet. Students will use OPIS to help them in answering these questions.

On launching OPIS, students are presented with the home screen (Figure 2), from which they can select the system they wish to investigate. Students can use the controls in the upper-right to (in left-to-right order of controls):

- Open a help menu

[4] i.e. Eyes on Exoplanets and Orbit Viewer.

- Toggle the appearance of OPIS between light and dark modes
- Open the settings menu to gain access to general and accessibility settings
- View OPIS in fullscreen

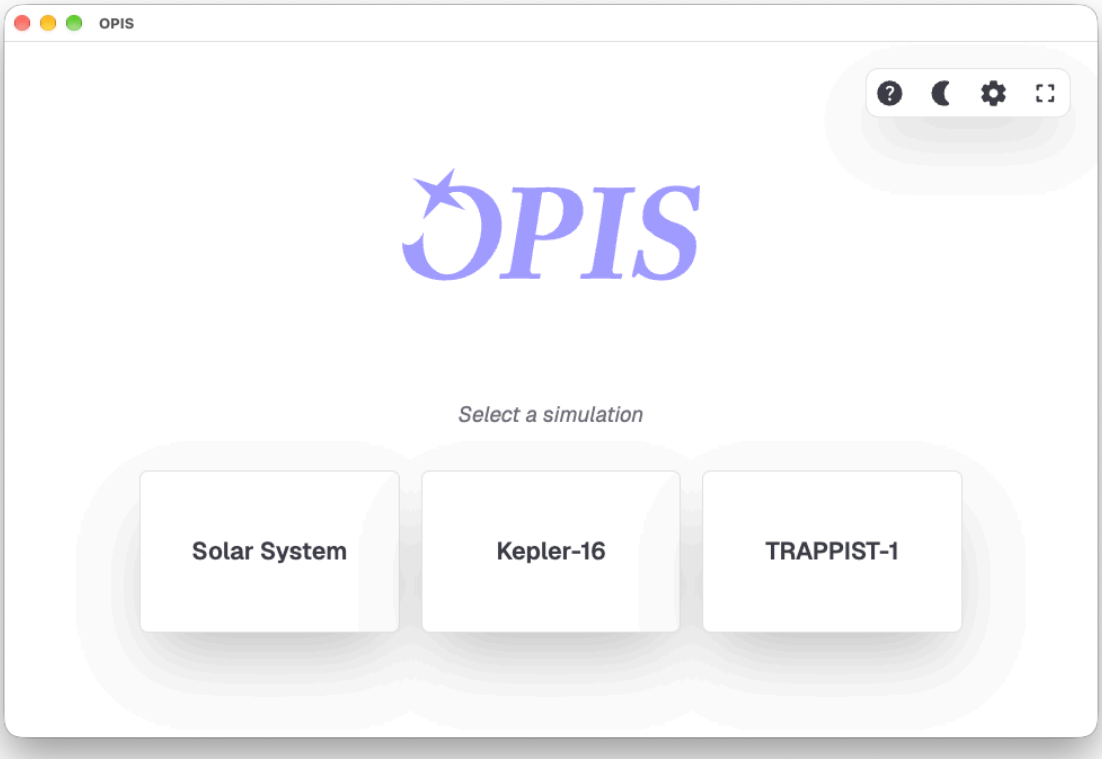


Figure 2
The home screen of OPIS. Students can select which system they wish to simulate, or using the controls in the top-right, display the help walkthrough, adjust OPIS' appearance, open the settings menu, and view OPIS in full screen.

After selecting the correct system, students enter the simulation page, which they use to answer the questions within the lab worksheet.

> *Which planet moves fastest around the Sun?*

> *Which planet has the longest year?*

To answer these questions, students will select the ‘Solar System’ option from the home screen. After being directed to the simulation page for the Solar System, students can press the play/pause button, circled in Figure 3. Once the simulation is playing, students can observe the relative speeds of the objects in the Solar System.

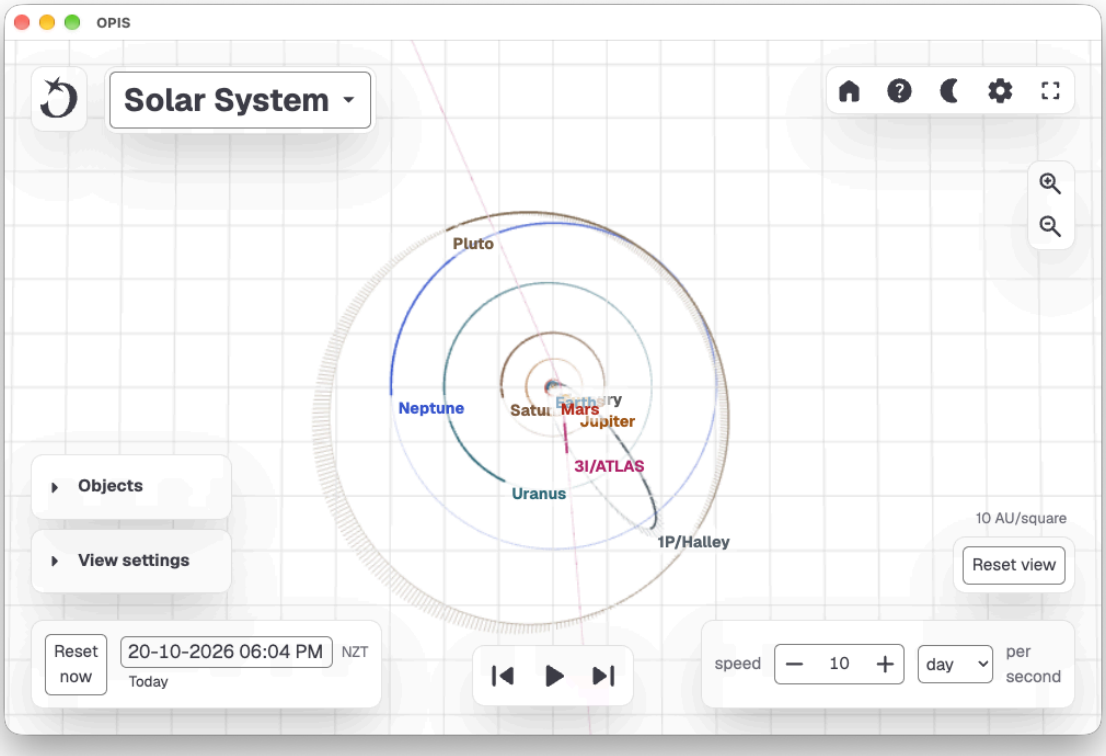


Figure 3

Students can use the play button in the bottom-centre of the page to play and pause the simulation.

> *How does [the orbit of Pluto] compare to the orbits of the planets [in the Solar System]?*

> *How does [the orbit of Halley’s Comet] compare to the orbits of the planets [in the Solar System]?*

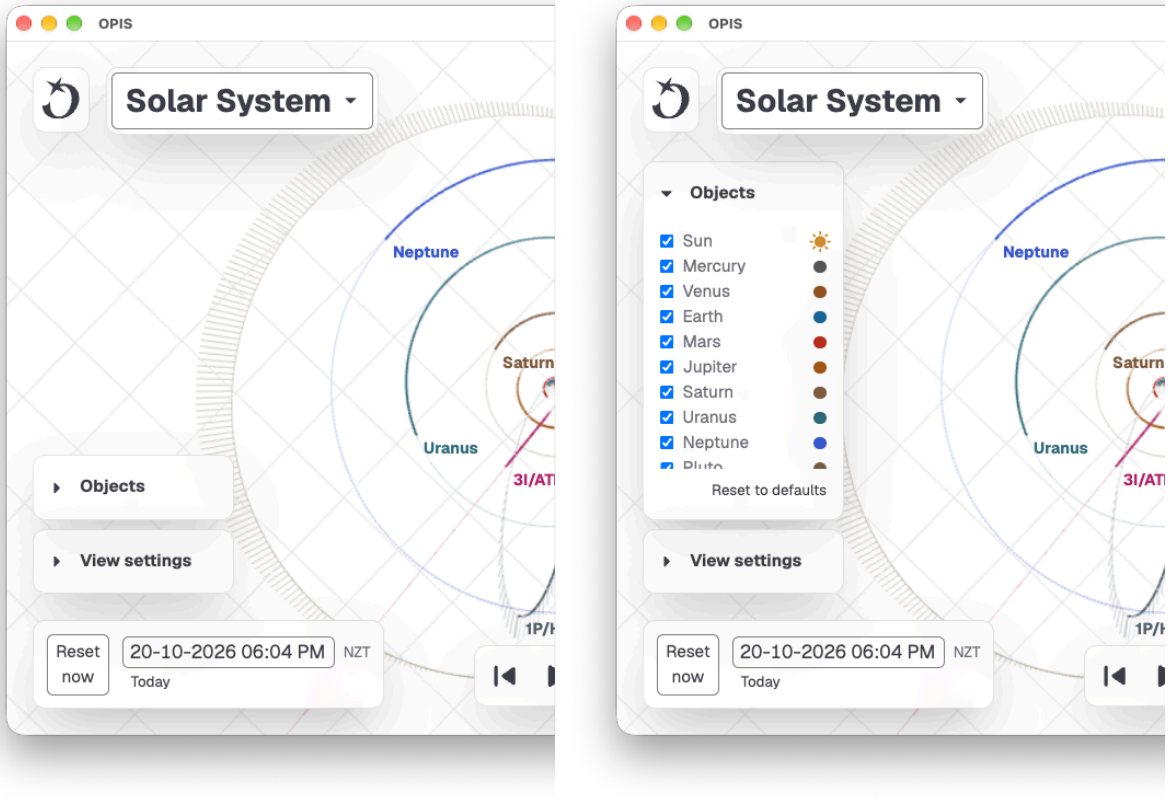


Figure 4

To show/hide objects such as Pluto, Halley's Comet, and ATLAS, students can first open the Objects *side panel, and then select the object(s) they wish to show/hide.*

> *How does [the orbit of 3I/ATLAS Comet] compare to the orbits of the planets [in the Solar System]?*

To answer these questions, students can add Pluto, Halley's Comet, and ATLAS by expanding the *Objects* side panel and checking the visibility for the objects, as shown in Figure 4. Then, users can rotate around the simulation as shown in Figure 5 to see that their orbits are inclined relative to the reference plane.

> *Is Pluto always the most distant object from the Sun in this simulation?*

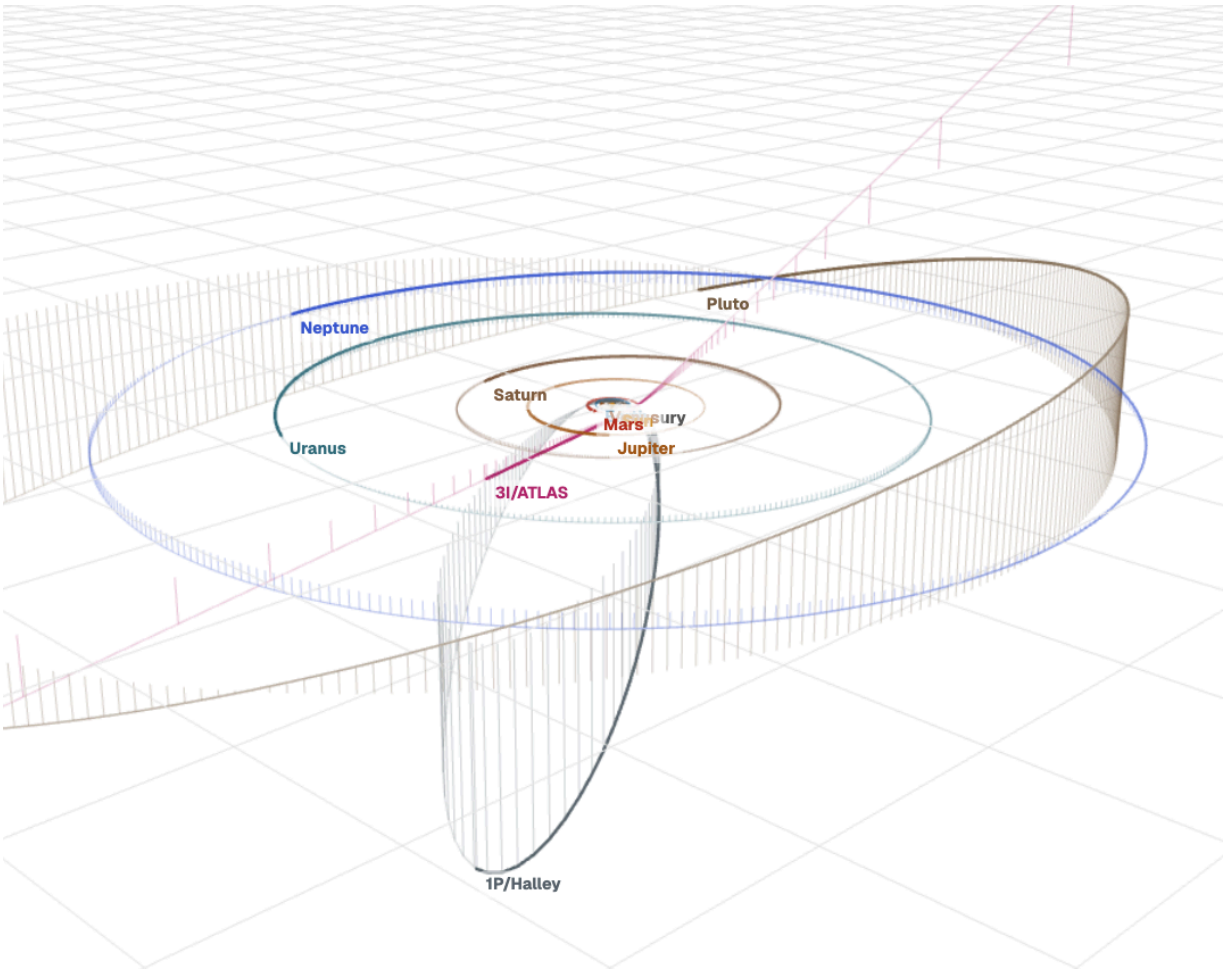


Figure 5

Users can rotate the simulation to show the inclinations of objects.

> *When does [Halley's Comet] move the fastest?*

> *Is [3I/ATLAS] part of the Solar System?*

These questions can best be answered by showing the 'Projection lines' setting under the *View settings* panel. Under this setting, lines are projected from the orbit lines to the reference plane, shown in Figure 6. From here, students can rotate and pan around, and play the simulation to see that:

- Part of Pluto's orbit comes closer to the Sun than that of Neptune
- Halley's Comet moves the fastest near its closest approach to the Sun (its perihelion)

- 3I/ATLAS is not part of the Solar System (as it has a hyperbolic orbit)

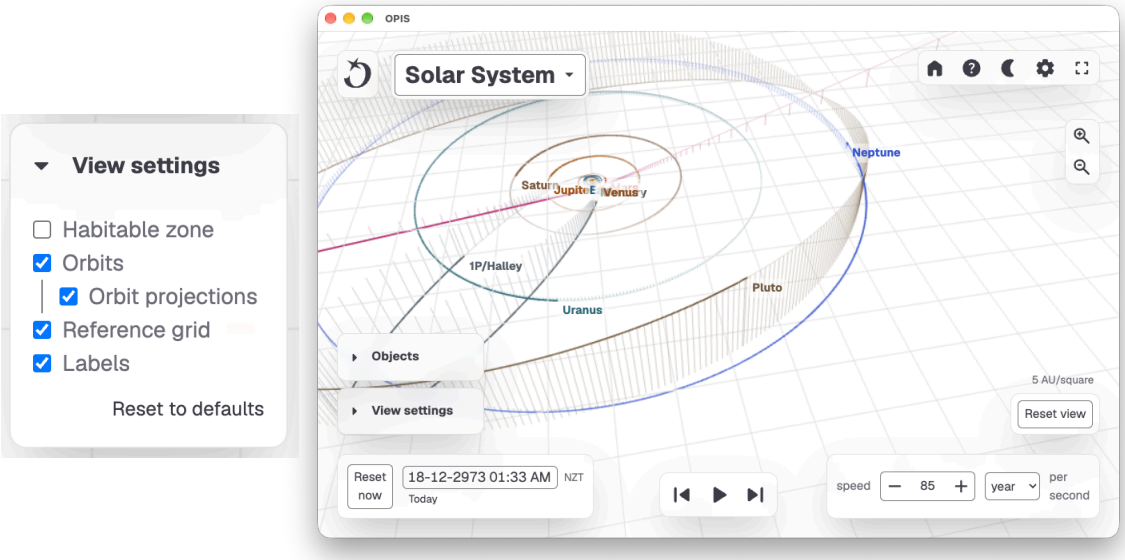


Figure 6

Left: the 'Projection lines' view setting in the View settings *side panel. Right: using the 'Projection lines' setting to see that part of Pluto's orbit comes closer to the Sun than Neptune's.*

> *How is [Kepler-16] different from our Solar System?*

> *What is Kepler-16 b and what is it orbiting?*

To answer these questions, students will switch to viewing Kepler-16 using the system picker dropdown shown in Figure 7.

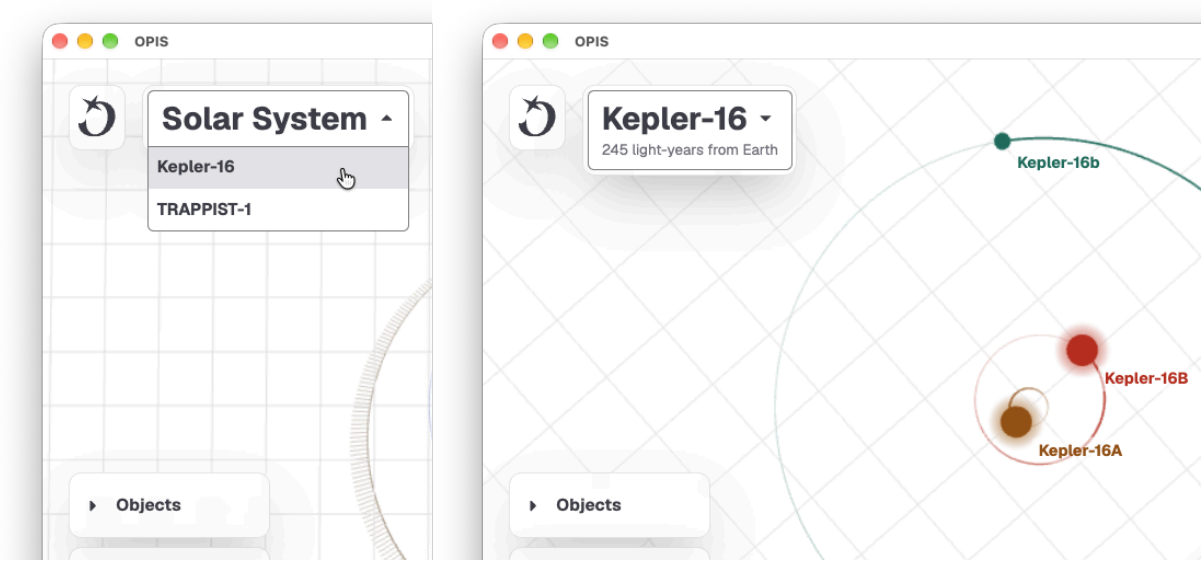


Figure 7

Students can select the system picker dropdown to switch between systems. Once viewing Kepler-16, students can see that Kepler-16b is a planet orbiting the binary star system Kepler-16 (comprising Kepler-16A and Kepler-16B).

After switching to viewing Kepler-16, students can see that:

- It is a binary star system (i.e. there are two stars in the system)
- Kepler-16b is a planet orbiting the binary stars Kepler-16A and Kepler-16B

> *Could liquid water exist in any of [the exoplanets in TRAPPIST-1]? [...]*

To answer this question, students can switch to viewing TRAPPIST-1, and show the habitable zone overlay in the *View settings* side panel, as shown in Figure 8.

This overlays the habitable zone, adjusted for each system, on the simulation, so that students can investigate which planets are within the habitable zone, and thus have a possibility of the existence of liquid water.



Figure 8

Students can enable show the habitable zone overlay in the View settings *side panel.*

> *[...] Compare [TRAPPIST-1] with our Solar System. Would the Earth be a habitable planet in [TRAPPIST-1]?*

To answer this question, students will use the *Compare to Solar System* feature presented in all exoplanetary systems (i.e. TRAPPIST-1 and Kepler-16), as shown in Figure 9.

With this overlay enabled, students can see the orbits of the Solar System overlaid atop the orbits of the current system. In this view, students can see that the orbit of the Earth is well outside of the habitable zone of TRAPPIST-1, and so students can deduce that Earth would not be a habitable planet in TRAPPIST-1.

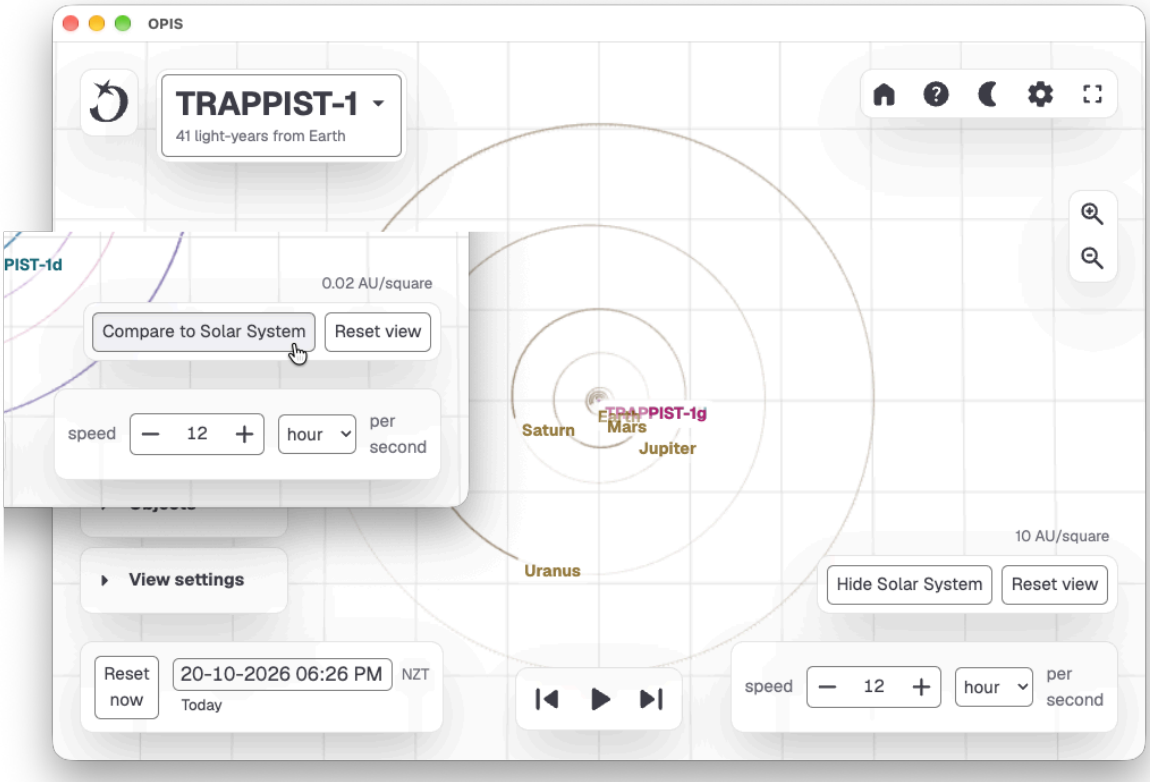


Figure 9

Students can click the Compare to Solar System/Hide Solar System *button to toggle the Solar System overlay. This overlays the orbits of the Solar System atop the current planetary system, which allows students to compare the orbits and scales of the two systems. It also allows students to see which (if any) planets of the Solar System are within the current system's habitable zone.*

The habitable zone of TRAPPIST-1 is barely visible as a small green ring in the centre, while the orbit of Earth can be seen well outside of the habitable zone.

6.2.2. ASTRO 100 tutor

ASTRO 100 tutors will assist students in using OPIS during lab sessions. They will guide students in how to use OPIS to answer the worksheet questions, should the students become stuck or have difficulties using the application.

Tutors will be familiar with OPIS in order to effectively assist students. In this way, we have designed OPIS to be as simple and intuitive to use as possible.

At the start of labs, tutors will instruct students how to launch OPIS. So that students can quickly get started with the application, they will give a brief overview on the basic functionality on how to use OPIS, such as:

- Selecting the system.
- Zooming and rotating around the simulation.

Should any students still require help, tutors can proceed to assist them individually.

6.2.3. Maintainers

ASTRO 100 tutors and staff members may wish to add more systems to OPIS outside of the three already included. This may be because the lab or course content is being updated or changed. This can be easily done by updating the `config.json` file with the parameters of the new system.

For each system the parameters outlined in Table 2, Table 3, and Table 4 are required.

Listing 1 shows an example snippet of the `config.json` file, to provide an idea of how maintainers can extend OPIS with additional planetary systems.

Table 2

The JSON schema for the representation of a planetary system in the `config.json` file.

JSON key	Description	Example value
`name`	Name of the system.	`"Kepler-16"`
`distance`	Distance from the system to the Solar System, in light years.	`245`
`objects`	Dictionary of objects (see Table 3 for schema).	`{ "Kepler-16A": {...}, ... }`

JSON key	Description	Example value
`timestamp`	Timestamp/epoch of the orbital parameters provided for this system, in Julian date[5] .	`2455212.12316`
`habitable zone`	Object with keys `start` and `end` representing the start and end of the system's habitable zone, in astronomical units.	`{ "start": 0.4, "end": 0.757 }`
`orbits`	Dictionary of orbits (see Table 4 for schema).	`{ "Kepler-16A": { ... }, ... }`

Table 3

The JSON schema for the representation of an object (of a planetary system) in the `config.json` file.

JSON key	Description	Example value
`colour`	Object representing the dark and light mode colours of the rendered object markers and orbit lines.	`{ "dark": "#d88379", "light": "#a93d31" }`
`type`	The type of the object.	`"star"`
`mass`	Object representing the object's mass and uncertainty, in kilograms.	`{ "value": 1.786e29, "uncertainty": 4.575e27 }`

[5] https://en.wikipedia.org/wiki/Julian_day

JSON key	Description	Example value
`temp`	Object representing the object's mean temperature and uncertainty, in Kelvin.	`{ "value": 2566.0, "uncertainty": 26.0 }`
`period`	Object representing the object's heliocentric and barycentric orbital periods, in seconds.	`{ "heliocentric": 7600506, "barycentric": 7779139 },`
`radius`	Object representing the object's radius and uncertainty, in metres.	`{ "value": 8.293e7, "uncertainty": 9.044e5 }`

Table 4

The JSON schema for the representation of an object (of a planetary system)'s orbit in the `config.json` file. For more information, see REBOUND's orbital elements documentation[6] .

JSON key	Description	Example value
`m`	Mass of the object, in kilograms.	`4.13610554870027e-6`
`P`	Orbital period of the object, in days.	`1.510826`
`e`	Eccentricity of orbit.	`0.0030547340309755285`
`omega`	Argument of pericenter of orbit, in $[0, 2\pi)$.	`2.3515648936386335`
`M`	Mean anomaly of orbit, in $[0, 2\pi)$	`0.806833482602799`
`inc`	Inclination of orbit, in $[0, \pi)$	`1.5707963267948966`

[6] https://rebound.hanno-rein.de/particles/orbitalelements/

JSON key	Description	Example value
`Omega`	Longitude of ascending node of orbit, in $[0, 2\pi)$.	`0`

Listing 1

A JSON snippet from the `config.json` file, indicative of how systems are represented in OPIS.

```
{
  "systems": [
    {
      "name": "Kepler-16",
      "distance": 245,
      "objects": {
        "Kepler-16A": {
          "colour": { "dark": "#e1812e", "light": "#945015" },
          "type": "star",
          "mass": { "value": 1.372e30, "uncertainty": 6.962e27 },
          "temp": { "value": 4450, "uncertainty": 150 },
          "period": { "heliocentric": null, "barycentric": 3549292 },
          "radius": { "value": 4.52e8, "uncertainty": 9.05e5 }
        },
        ...
      },
      "timestamp": 2455212.12316,
      "habitable zone": { "start": 0.4, "end": 0.757 },
      "orbits": {
        "Kepler-16A": {
          "m": 0.6897
        },
        "Kepler-16B": {
          "m": 0.20255, "a": 0.22431, "e": 0.15944, "inc": 1.5767321916,
          "Omega": 0.0, "omega": 4.5983142605, "M": -2.9864677897
        },
        "Kepler-16b": {
          "m": 0.0003178778, "a": 0.7048, "e": 0.0069, "inc": 1.5713583228,
          "Omega": 0.0000523599, "omega": 5.5501470213, "M": -3.6912491949
        }
      }
    },
    ...
  ]
}
```

6.3. Design

6.3.1. UI/UX

Jennifer Butcher

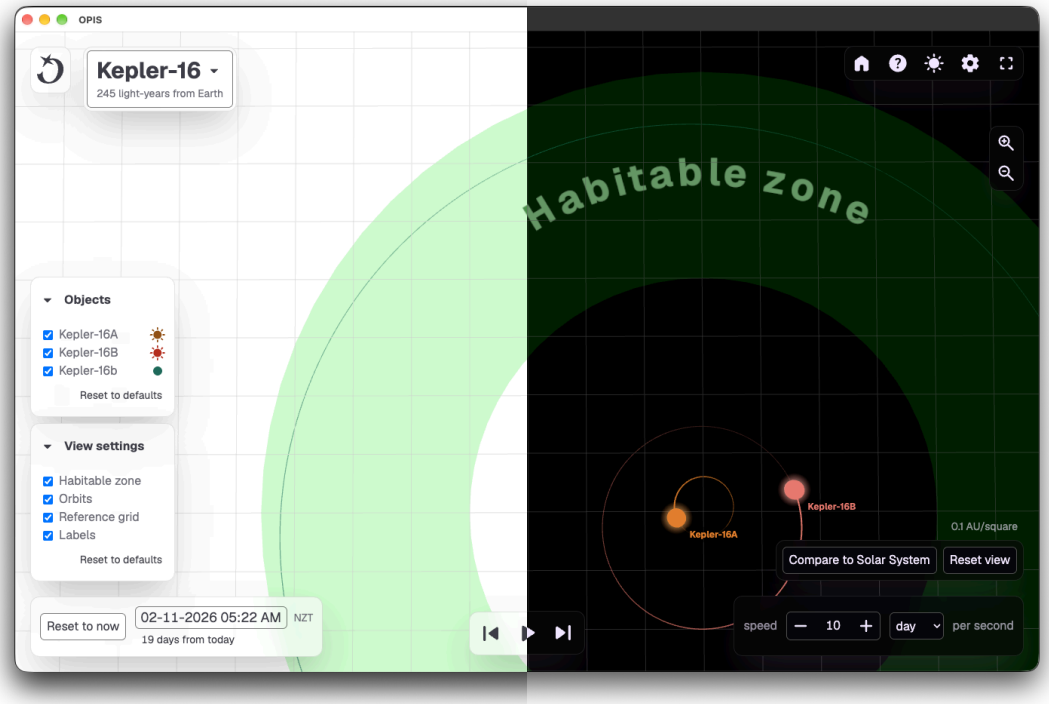


Figure 10

Composite of OPIS' simulation UI in light mode (left) and dark mode (right).

The OPIS interface is designed to keep the 3D simulations as the primary focus of the application. To support this, the interface has a minimal visual style, ensuring that the necessary information and controls remain clear and accessible, while limiting unnecessary visual elements. This ensures students can concentrate on exploring and interacting with the simulations, without the interface competing for their attention.

OPIS uses a limited colour scheme, available in both light and dark modes (Figure 11, Figure 12). This allows users to select an appearance suited to their preferences and viewing conditions. To accommodate users with colour vision deficiencies, information and functionality are never distinguished by colour alone.



Figure 11

Light mode colour scheme. L-R: text, secondary, primary, accent colours.

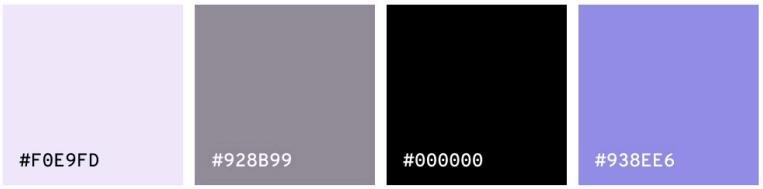


Figure 12

Dark mode colour scheme. L-R: text, secondary, primary, accent colours.

Font selection is similarly prioritised for clarity and readability. Geist is used as the default application font for its clean, legible letterforms, while OpenDyslexic is available as an alternative to accommodate users who may benefit from different letterform designs, particularly those with dyslexia or other reading difficulties.

The interface is constructed from reusable components, ensuring consistency in the appearance and behaviour of controls throughout the application. In alignment with the identified user requirements, no assumption is made about the user's familiarity with technical interfaces. Interactive components are all accompanied by descriptive tooltips explaining their functions, reducing the need for prior knowledge or trial-and-error.

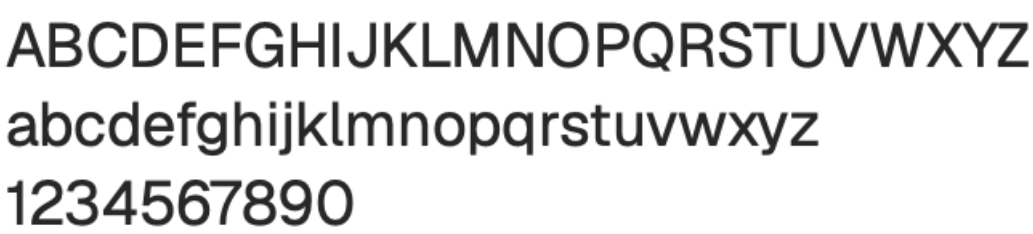


Figure 13

Geist in regular weight. Geist is used for all body text within OPIS.

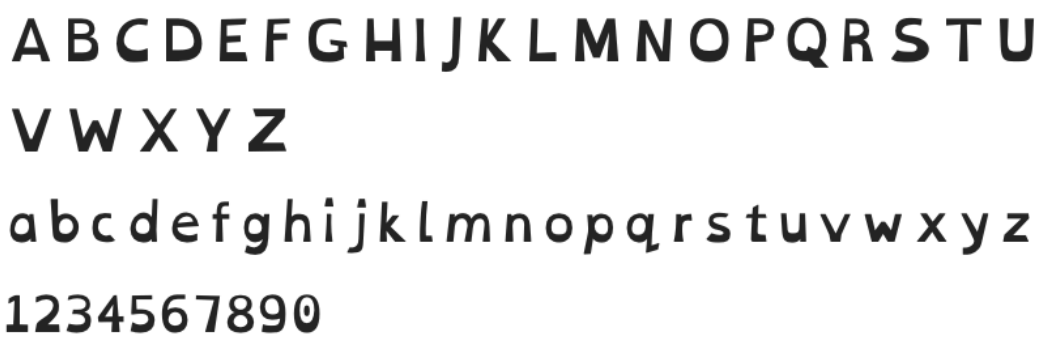


Figure 14

OpenDyslexic in regular weight. OPIS offers OpenDyslexic as a font option for users with reading difficulties.

Additionally, all functionality required to complete the lab worksheet is accessible directly from the simulation page, rather than being hidden within nested menus. This makes essential controls readily discoverable, and minimises the steps required to work through the lab activities.

Further control and customisation options are available without interfering with the simulation controls. A toolbar provides access to help documentation, light and dark mode switching, and full-screen display. A settings overlay allows users to switch the application font, adjust text size and simulation marker size, and toggle the style of orbit paths. These options allow

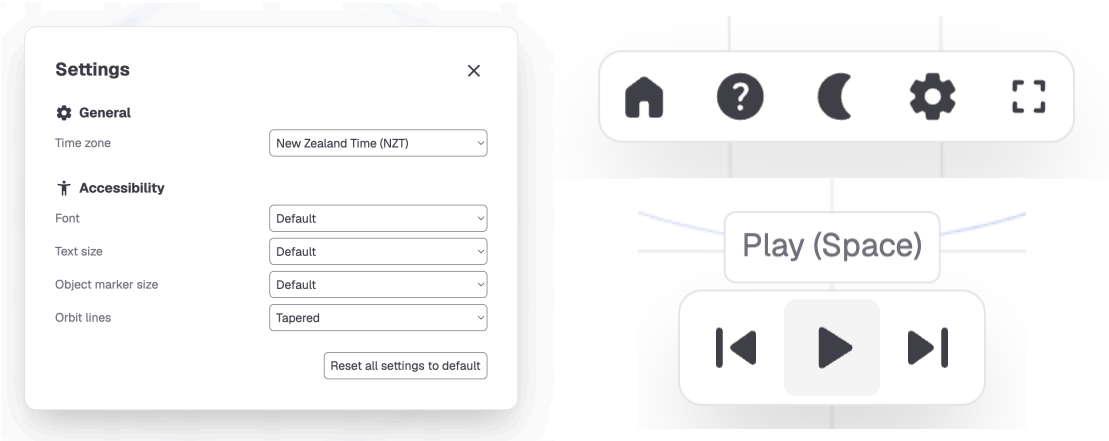


Figure 15

Clockwise from top-left: settings menu; toolbar; tooltips (also showing keyboard shortcuts).

students to adapt the interface to their individual preferences and accessibility needs, while preserving the primary functionality of the main interface. All these controls are shown in Figure 15.

Keyboard shortcuts are also available for common actions and are indicated within the relevant tooltips. For example, the playback of the simulation can be started or stopped using the space bar.

Although OPIS is primarily designed for use on the ASTRO 100 laboratory computers, its cross-platform support also allows students to access the application on their personal devices. Thus, to accommodate different screen sizes, OPIS supports a minimum window of 800×600px. Interface elements scale to fit the available space where possible, while overlays become scrollable when their controls cannot be displayed in full. This ensures that controls and information remain accessible across supported screen sizes, without compromising the usability of the simulations.

6.3.2. System architecture

Elle Fleming

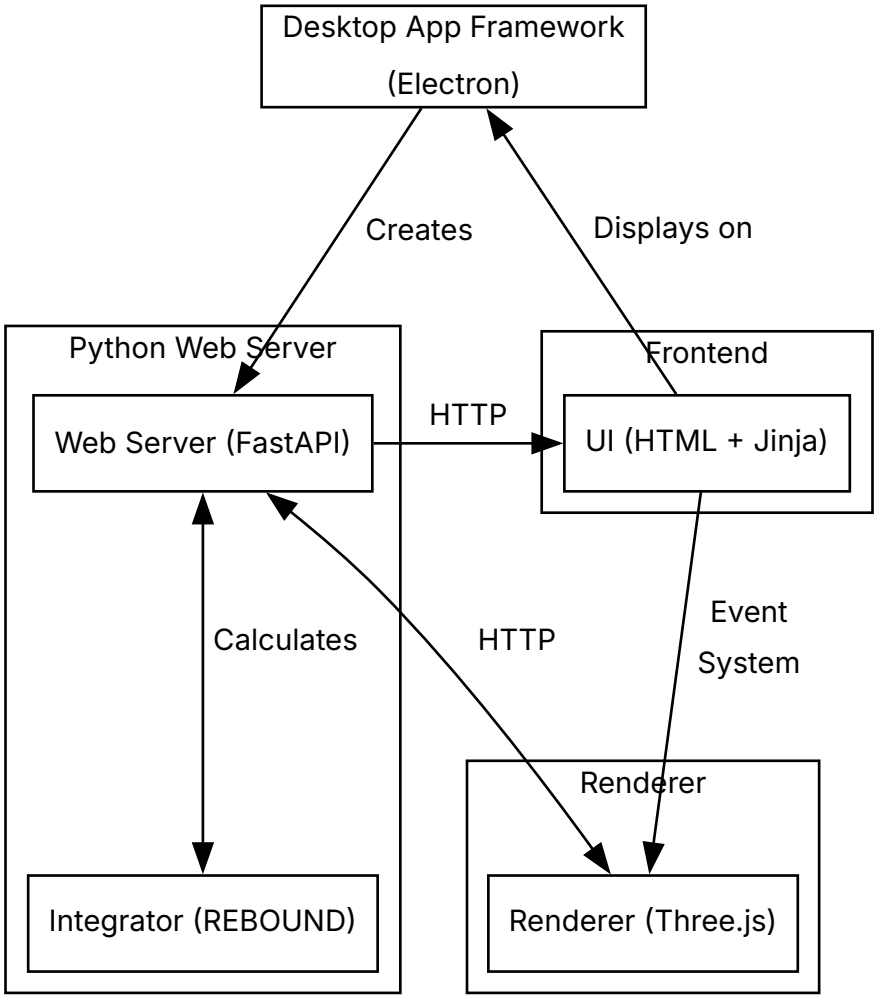


Figure 16

Module-connector summary diagram of OPIS

OPIS can be classified into four main modules: the desktop application, backend, frontend, and renderer. These modules are designed to be as independent as possible to anticipate for possible future modifications. We chose this as we were not initially set on the technologies we chose and needed to ensure that, if needed, we could pivot. An example of this is Electron is only ever interacted with via well-defined inter-process communication (IPC)[†] and thus could be replaced with another framework should there be any issues.

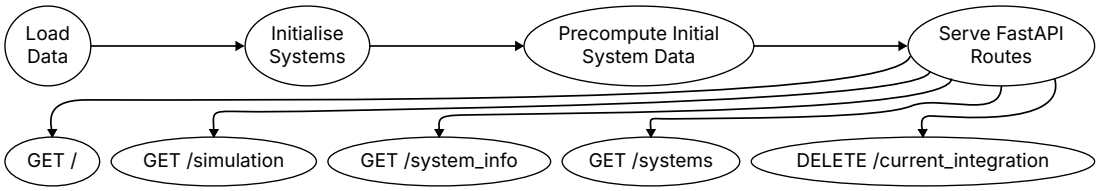


Figure 17

Backend startup state overview

Throughout the application, when crossing boundaries between modules, we used abstractions to ensure separations of concerns. IPC† is used in the boundaries of Electron-backend, backend-renderer and backend-frontend so that it is easy to change one module without changing the other. The boundary between the renderer and UI was abstracted through the event system as discussed in Section 6.3.6 (Event system) and in more detail in Section 7.4 (Event system).

6.3.3. Backend

Elle Fleming

The design of the backend architecture is uncomplicated compared to other parts of the system. It only has five non-file-serving endpoints as discussed in more detail in Section 7.6.1 (FastAPI and Uvicorn). Python only comprises a single logical module.

The reason that there is a lack of separation into modules is because four of the endpoints depend on similar or the same logic/calculations leading to tight coupling. We also designed the API to be as simple as possible to minimise the IPC† surface area between the backend and renderer. This helps reduce bugs and mismatching behaviour since no endpoints duplicate any behaviour. For example we determined it was best that `GET /systems` took a query parameter list `system_names` for returning multiple systems data rather than split the endpoint into a separate `GET /compare_to_solar_system`. This also allowed anticipation of future modifications as if we needed compare or get multiple systems' data (other than the Solar System) the endpoint already exists and the functionality is there.

6.3.4. Renderer

Jianing Li, Ethan Gee

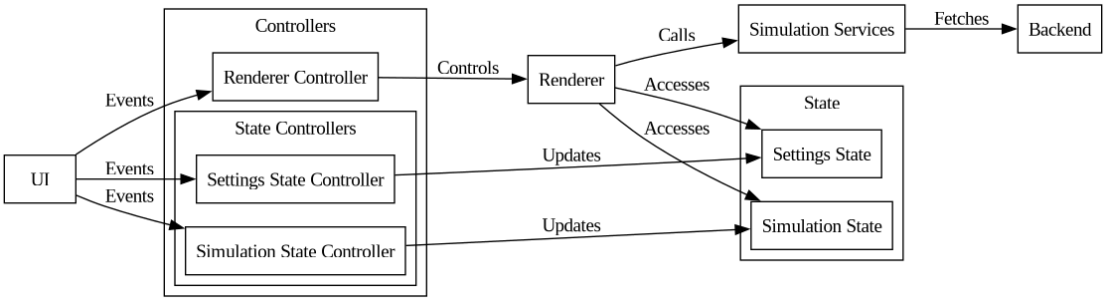


Figure 18

Renderer component and connector diagram

The simulation renderer architecture consists of four key components: the controllers, state, services, and the renderer itself.

The controllers provide an interface for interacting with the renderer and state, and contain a set of listeners for events emitted by the UI, as discussed in Section 6.3.6 (Event system). This was implemented as a separate layer from the renderer to isolate the handling of events from the renderer logic.

To handle user interactions, the renderer exposes an interface to the renderer controller, and would trigger an immediate update to the simulation. These include functions for stepping through time, setting the theme, and toggling the visibility of objects. The renderer also requires access to simulation state for the current application session as well as the persisted settings state, discussed in Section 6.3.5 (State management). This state is accessed by the renderer on initialisation and throughout its lifetime, so that the simulation displays correctly and reflects any state updates, and are managed by the state controllers.

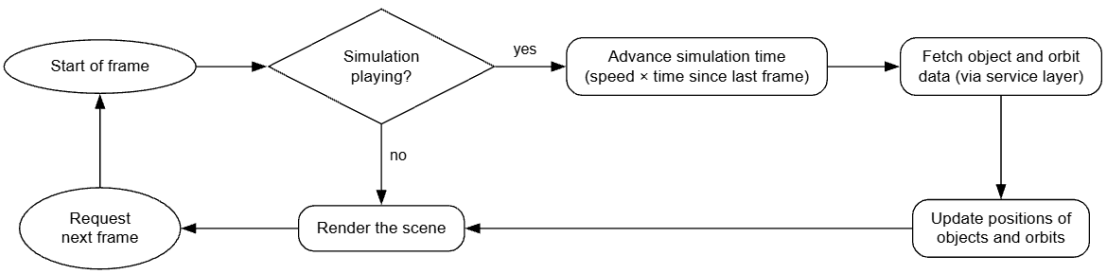


Figure 19

Render loop

On initialisation, the renderer calls the service layer to fetch the initial data for the current system, including the inner and outer habitable zone radii and the data for the objects and orbits at the initial simulation time. These are then used to create and position the objects, orbits, and habitable zone in the scene.

As seen in Figure 19, on each frame while the simulation is playing, the simulation time is advanced based on the simulation speed, the object and orbit data at this new time is fetched, and the positions of objects and orbits are updated, before the scene is rendered again. While the simulation is paused, the scene is still rendered every frame so that it updates as the user moves or rotates the camera. All backend requests are made through the service layer, which separates them from the renderer logic.

Initially, our main concern with this design was the potential for performance degradation from the amount of data requested from the backend on every frame. As shown in Listing 1, objects and orbits require a large amount of data to describe. To address this, we considered limiting the data requested from the backend by interpolating the positions of objects and orbits from the data already received. Instead of fetching new data on every frame, the renderer would fetch data at less frequent intervals and use the fetched values to estimate the positions of the objects and orbits for the frames in between. However, upon experimentation on the lab computers, we found that this data load did not have a significant impact on performance. Moreover, when experimenting with interpolation, we found that it gave minimal performance gain, while the interpolated positions deviated significantly from the actual positions, especially at high simulation speeds. Given this, we opted for the simpler approach of fetching the data on every frame.

6.3.5. State management

Ethan Gee

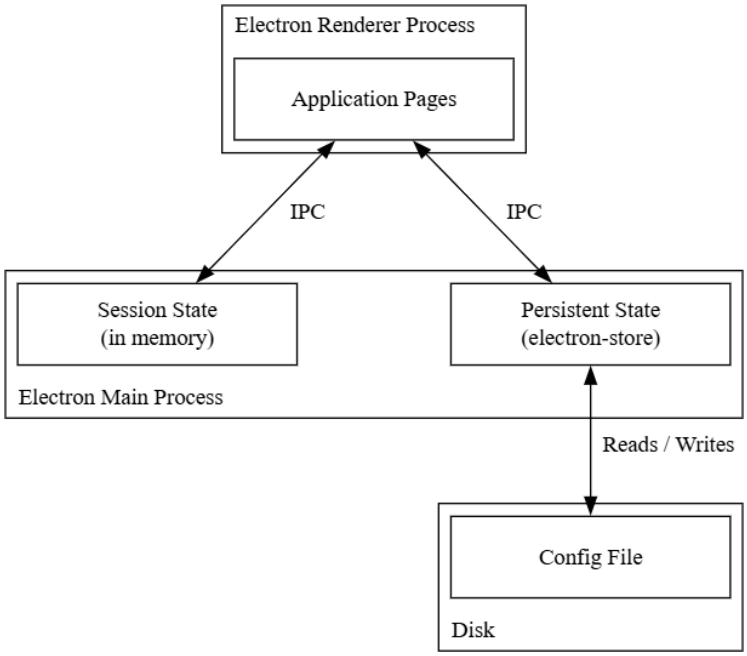


Figure 20

State management component and connector diagram

Our application needs to retain certain state, such as user preferences and the progress of the simulation, so that it is not reset when the user navigates between pages or reopens the application. However, how long state needs to be retained depends on what it applies to.

For example, the application theme applies regardless of the simulation being run, so it should be restored the next time the application is opened. Conversely, the simulation speed only applies to the simulation currently being run, so there is no need to restore it when the application is reopened and a new simulation is started. As such, our state management component is designed to manage both session and persistent state, as seen in Figure 20.

Session state is maintained within a single application run and is lost when the application is closed. Persistent state is maintained between application runs, and remains available even if the user restarts their computer.

An Electron application has a single main process, which stays alive for as long as the application is open, and each application window has its own renderer process which displays the application's pages. As state held in a

page is lost when that page is unloaded, we hold session state in the main process, which retains it when the user navigates between pages. Persistent state is written to a configuration file on disk using Electron Store[7] , a library for persisting data in Electron applications. To read and write both types of state, the pages use IPC† to communicate with the main process, which handles the storage on their behalf. The advantages of this design are that all pages share a single copy of the state, providing the application with a single source of truth. The pages are also decoupled from how state is stored, so the underlying storage method can be replaced without affecting the pages.

6.3.6. Event system

Ethan Gee

The UI, renderer, and state management components need to communicate with one another to share data and invoke actions. Our initial design had each component expose an interface for for the others to call directly. For example, pressing the simulation play button would have the UI call the renderer directly to start the simulation. However, this couples each component to the interfaces of the other components, so a change to one interface would require a

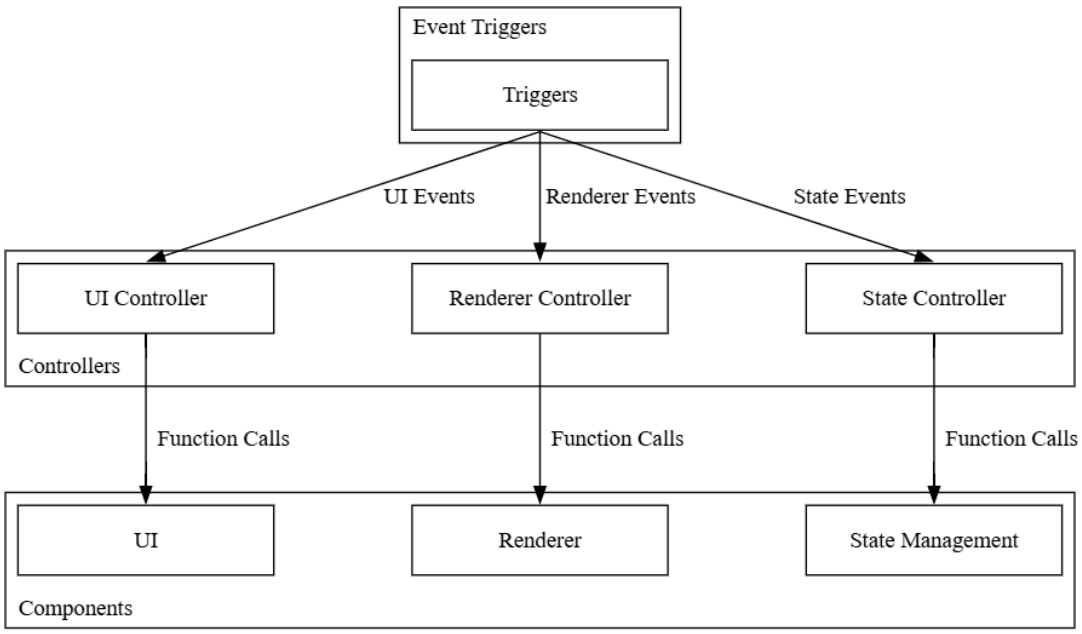


Figure 21

Event system component and connector diagram

[7] https://www.npmjs.com/package/electron-store

corresponding change to every component that uses it. It also scales poorly, as every new interaction would add another call path that must be maintained. We therefore decided against this design and opted for an event system instead, which decouples the components by having them communicate through events.

Our event system is organised in three layers, as shown in Figure 21. The first layer consists of event triggers, which publish custom events when a specified action occurs, such as the user clicking a button. Events can optionally carry data, such as the new value when the simulation speed is changed. The second layer consists of controllers, one each for the UI, renderer, and state management components. Each controller subscribes to the events relevant to its component and invokes the necessary functions of that component. A single event can be subscribed to by multiple controllers, so a single action can update several components at once. The third layer consists of the components themselves: the UI presents information to the user, the renderer runs and renders the simulation, and state management maintains the state of the application.

As an example, when the user presses the simulation play button, a trigger publishes a play event. The renderer controller then invokes the renderer to start the simulation, and the UI controller invokes the UI to change the button's appearance to a stop button.

The advantages of this design are modifiability and extensibility. Calls into the components are confined to the controllers, so a change to a component's interface only requires updating its controller. The system is also easy to extend. Supporting a new component requires adding a controller, along with any new triggers and events it needs, and supporting a new interaction requires adding a trigger and having the relevant controllers subscribe to its events. In both cases, the existing triggers and the other components are unaffected. The primary tradeoff is the overhead of routing communication through events, though we deemed this insignificant as it has a negligible effect on performance and adds no noticeable delay for the user.

6.3.7. Distribution and updates

Ethan Gee

OPIS supports Windows, macOS, and Linux. To avoid running our build process manually on each operating system for every release, we use a continuous

deployment (CD) pipeline, which builds the installers and publishes them as a release on our repository's GitHub releases page[8] . Our CD pipeline is triggered by pushing a tag to our repository. The advantages of this are that we do not require access to a Windows, macOS, and Linux to build each installer when publishing a release, and that maintainers can publish a release without knowing how our build process works, as they only need to trigger the CD pipeline.

OPIS also includes an auto-updater, which notifies users when a new version of OPIS is available. When OPIS is launched, the auto-updater checks whether the latest version on our GitHub repository is newer than the installed version. If it is, the user is prompted to download the new version. If they decline, they are not prompted again until the next time they launch OPIS. If they accept, the new version is downloaded, and the user is then given the choice of restarting OPIS now or later to finish the update. This makes it easy for users to keep their installed version of OPIS up to date. Updating OPIS is also seamless, taking less than a minute on the lab computers.

[8] https://github.com/uoa-compsci399-s2-2026-anna/team-9-project/releases

7. Implementation

7.1. UI/UX implementation

OPIS' frontend was built using HTML and Jinja templating for markup, Tailwind CSS for styling, and JavaScript for interactivity. For the most part, implementation was relatively straightforward. However, as our familiarity with these frameworks developed alongside the project, we realised some early implementation decisions were inefficient. This prompted several rounds of restructuring and refinement, discussed further in Section 7.1.1 (Markup) and Section 7.1.2 (Calendar).

7.1.1. Markup

Jennifer Butcher, Derek Chai

All pages in OPIS extend a single base template, `base.html`, which holds the default styling, fonts, theme handling, and components shared across the application. The two other pages, `home.html` and `simulation.html`, extend it and supply only their own content. This ensures that any changes to the shared structure only needs to be made in one place.

Templates can include other templates (composites of macros and HTML) and other macros (parameterised HTML). This overall structure is shown in Figure 22.

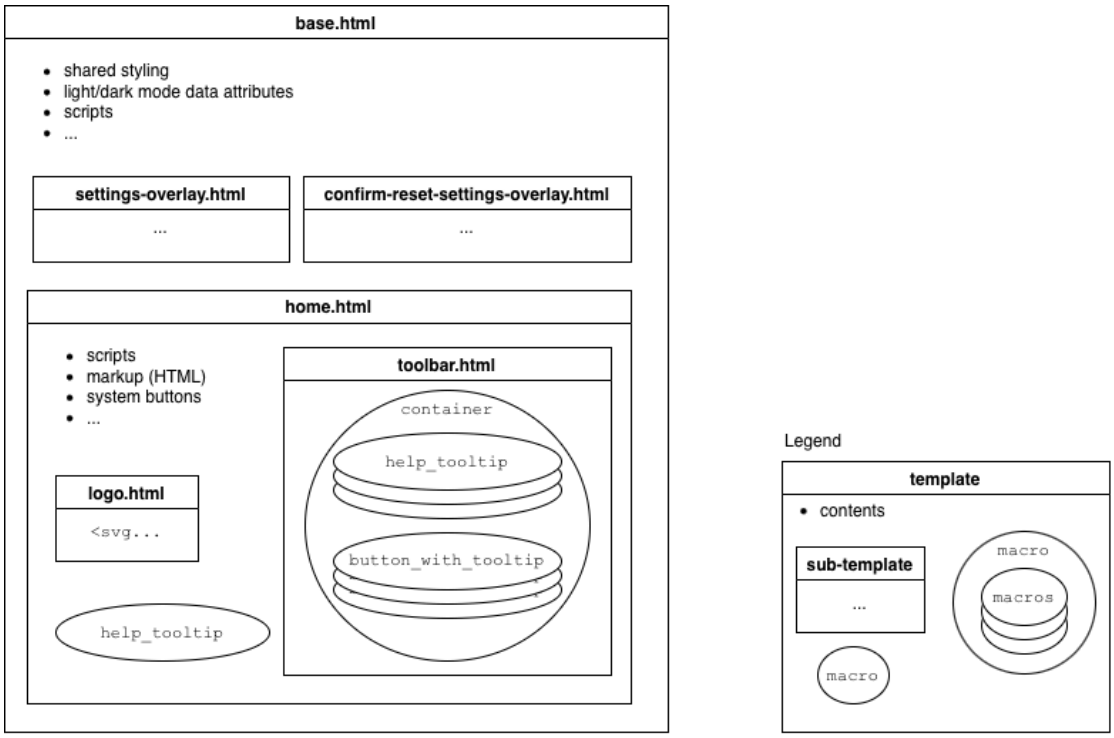
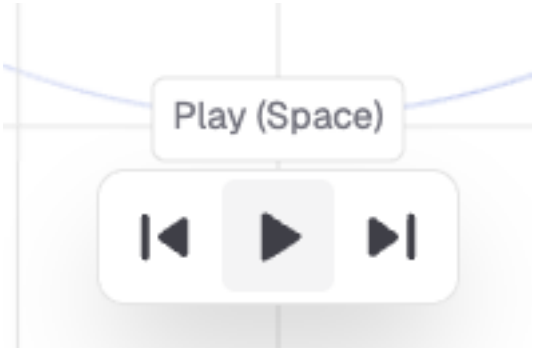


Figure 22

Diagram showing how the home page of OPIS is composed from templates, macros, and HTML. This is not an exhaustive representation of the true structure, but rather an indicative one which reflects the general hierarchy of macros and templates used to compose the home page.

Initially, our markup consisted of only three templates: `base.html`, and the two pages that extend it. As we became more familiar with Jinja, we iteratively broke these templates down into smaller and smaller components and macros, in order to remove repeated markup. This meant that new parts of the interface could be assembled from existing components rather than being written from scratch, and styling and behaviour of elements was consistent throughout the interface.

In this way, repeated parts of the interface are written once as Jinja macros and customised with parameters. For example, `button-with-tooltip` is a reusable icon button which shows a tooltip on hover. Its parameters set the icon, the tooltip's text, position and alignment, the events the button fires when clicked, and whether that event should repeat while the button is held down.




```
<!-- Play/pause button -->
{{
  button_with_tooltip(
    tooltip_id="play-pause-tooltip",
    svg_path_id="play-pause-svg-path",
    tooltip_position="above",
    svg_size=10,
    tooltip_text="Play",
    svg_path=...,

    on_click_events=["sim:toggle"]
  )
}}
```

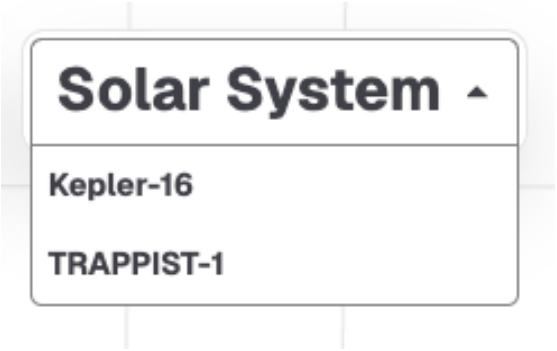
Figure 23

Left: the simulation playback controls. Right: Jinja macro `button_with_tooltip` used to display the play/pause button (right).

This is used in OPIS for controls such as the playback speed controls, such as shown in Figure 23.

Keyboard shortcuts for events are also displayed automatically within the corresponding buttons' tooltips. This is achieved by defining keyboard shortcuts centrally in the `shortcuts.json` file. The web server renders the Jinja template with the shortcuts passed as contextual data, and makes it available to all frontend templates. When a button is rendered, a generic JavaScript function checks if it fires an event present in `shortcuts.json`, and if so, appends the appropriate label to its tooltip.

The system selection buttons in `home.html` and the system dropdown in `simulation.html` also leverage configuration files to avoid duplicated HTML. The list of available planetary systems is read from `config.json`, and the `system_picker.html` iterates over this list to create each option in the system dropdown. This means adding a new planetary system only requires updating the configuration file, rather than modifying any of the markup itself.




```
<div class="flex flex-col">
  {% for system in dropdown_systems %}
    <a
      href="/simulation/
{{ system.name }}"
      class=...
    >
      {{ system.name }}
    </a>
  {% endfor %}
</div>
```

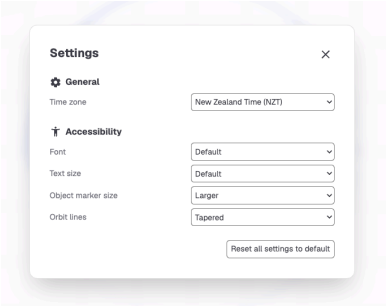
Figure 24

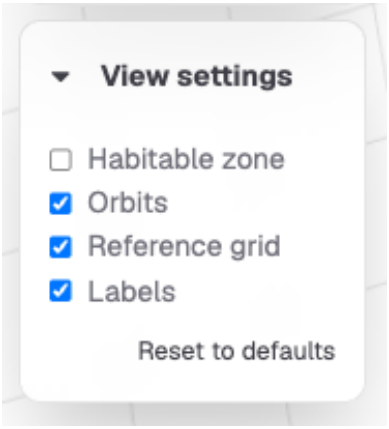
Left: the rendered system picker dropdown. Right: Jinja template code for rendering each option in the system picker dropdown.

Some other macros that are used widely throughout the application are shown in Table 5.

Table 5

Detail of macros which are reused throughout OPIS.

Macro	Description	Example
`overlay`	Reusable overlay for settings menus and other modals	

Macro	Description	Example
`side_panel`	Collapsible, scrolling side panel for *Objects* and *View settings*	
`toolbar`	Toolbar reused across templates	

7.1.2. Calendar

Jennifer Butcher

The frontend simulation datetime picker, used to view and change the simulation's current time, was initially built using the native HTML input, `<input type='datetime-local'>`. Although straightforward to implement, this had several limitations which conflicted with OPIS' requirements. Most notably, there was extremely limited control over appearance and behaviour. This meant we couldn't customise the picker's appearance to align with the rest of our application, or have it respond appropriately to settings such as font and font size.

OPIS also requires functionality beyond that of a conventional datetime input. The picker needs to display the simulation's current time in the user's selected timezone, update the displayed time as the simulation progresses, and allow users to jump to a new date and time within 10 years of the current simulation time. We therefore replaced the native input with Flatpickr[9] , a lightweight JavaScript datetime picker that provides greater control over appearance and interaction.

Although Flatpickr provided a more customisable starting point, it still did not support all of OPIS' requirements directly. In particular, it did not natively

[9] https://flatpickr.js.org/

handle the interpretation of selected dates in different timezones. We therefore implemented a conversion layer using the Luxon[10] JavaScript library, to manage the relationship between the simulation's timestamp and the date and time displayed to the user.

When the page first loads, `getSimulationTime()` retrieves the simulation time associated with the selected system. If a simulation time has previously been stored, that value is used; otherwise, the simulation defaults to the current date and time. The simulation time is represented internally as a UNIX timestamp[11] in milliseconds, corresponding to the number of milliseconds elapsed since 1 January, 1970 at 00:00:00 UTC. The `format_sim_date` function converts this timestamp into a date and time string in the user's selected timezone, which is then inserted into the calendar input as its initial value.

After initialisation, `currentSimulationTime` in `simulationRenderer.js` stores the simulation's current timestamp, which is updated as the simulation progresses. Luxon's `formatInTimeZone` function formats this timestamp for display in the picker.

When the user selects a new date and time, Flatpickr's `onChange` callback passes the selection to Luxon's `convertToEpoch` function. This interprets the selected date, time, and timezone, and converts it to a UNIX timestamp in milliseconds. The resulting timestamp is published through the application's event bus as an `EVENTS.SIM.CALENDAR_CHANGE` event. The corresponding event handler in `simulationRendererController.js` passes the timestamp to `setSimulationTimetoTime` in `simulationRenderer.js`, which updates the current simulation time.

The picker's minimum and maximum selectable dates are also updated relative to the current simulation time. These extend ten years in either direction, subject to the simulation's overall permitted time range (year 1 to year 9999).

Together, Flatpickr manages the calendar interface, while Luxon handles timezone interpretation and conversion between the date and time displayed on the front-end, and the timestamp used by the simulation.

[10] https://moment.github.io/luxon/
[11] https://en.wikipedia.org/wiki/Unix_time

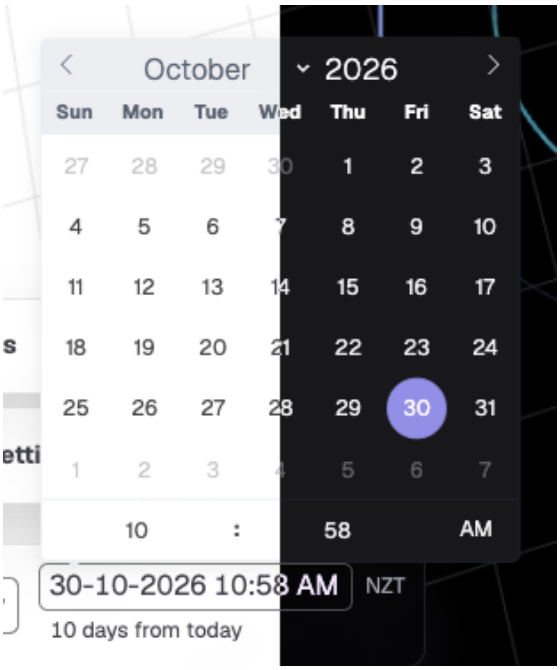


Figure 25

OPIS' calendar in light and dark modes, implemented using Flatpickr.

7.1.3. Styling

Jennifer Butcher, Derek Chai

We used Tailwind CSS to automatically generate the CSS stylesheet for OPIS. Tailwind uses custom classes such as `p-6` (padding) or `bg-white` (background colour) to automatically generate the appropriate CSS. This reduces the context switching between HTML and CSS experienced in traditional development. It also reduces the chance of unintended styling changes across the entire project as each class/style is localised to only one element.

Tailwind handles colours used in OPIS, and changing between light and dark modes. Tailwind defines the colours used in OPIS, and automatically generates required CSS for light and dark mode styling. This is done through a data attribute `data-theme='{{ 'dark' if settings.darkMode else 'light' }}'` on the base `<html>` tag of OPIS, which encodes the current dark/light mode state. Tailwind uses this to appropriately update styling as dark/light mode changes.

To implement reusable styling, 'component classes' defined in `input.css` are used, as shown in Listing 2. For example, the class `.bordered` give a standardised rounded border and outline, `clickable` adds hover and active styling that marks an element as interactive, and `shadowed` adds a drop shadow. Each class includes its dark mode variant (e.g. `dark:border-white/30`), so a

component only needs to include a single component class appear correctly in either theme.

Listing 2

Example of some Tailwind component classes used in OPIS. For example, `.clickable` adds hovering and active (actively being clicked) styling to buttons, which is responsive to dark and light mode.

```
@layer components {
  .bordered {
    @apply border border-black/50 dark:border-white/30;
  }

  .clickable {
    @apply hover:bg-zinc-100 active:bg-zinc-100 active:inset-shadow-sm
dark:hover:bg-zinc-900 dark:active:bg-zinc-700;
  }
  .shadowed {
    @apply p-1 shadow-2xl/50 shadow-black/50 dark:shadow-purple-300/30;
  }

  ...
```

Tailwind also provides the ability to add custom 'variants', which apply alternate styling based on a specific condition, such as a selector or media query. For example, we implemented a `short:` variant which applies some Tailwind styling onto an element based on the window size, shown in Listing 3.

Listing 3

Custom Tailwind variant `short` used in OPIS. This variant is enabled based on a media query of the screen height and width. Then, a style is applied on an element when the variant is enabled: for example, when the `short` variant is active (i.e. the screen size is smaller than 1010×700 pixels), the `<span>` is hidden.

In `input.css`:

```
@custom-variant short (@media (max-height: 700px), (max-width: 1010px));
```

In `text-button.html`:

```
<span class="short:hidden">{{ button_text }}</span>
```

Tailwind's variants also handle accessibility in the frontend. For example, selecting the accessible font OpenDyslexic activates variants which widen controls which would otherwise clip the wider letterforms.

7.1.4. Interactivity

Jennifer Butcher, Derek Chai

Interactivity in the OPIS interface is built around a separation between the controls that trigger actions, and the scripts that respond to them. Interface components such as buttons declare their behaviour through a combination of Jinja macros and HTML data attributes. JavaScript scripts query these shared data attributes to implement functionality, rather than registering individual event listeners for each control. This interactivity leverages the event system, discussed in greater detail in Section 7.4 (Event system).

For example, buttons declare the events they wish to publish in the `on_click_events` parameter to the button macro. The macro contains the data attribute `data-button-type` which is shared across all button macros (`button_with_tooltip` and `text_button`). This is shown in Listing 4.

Listing 4

Left: example of the call site of a button macro. Note the parameter `on_click_events`, where the event to be fired on click is passed. Right: the definition of the macro `button_with_tooltip`. Note the data attribute `data-button-type` on the parent `<div>`.

In `toolbar.html`:

```
{{
 button_with_tooltip(
   tooltip_text="Help",
   ...
   on_click_events=["toolbar:help_button_click"],
 )
}}
```

In `button-with-tooltip.html`:

```
{%
  macro button_with_tooltip(
    tooltip_text,
    svg_path,
    ...
    on_click_events=[],
    ...
  )
%}
  <div
    class=...
    data-button-type ...
```

Finally, a JavaScript script queries all elements with selector `[data-button-type]` and registers event listeners to publish the events listed in the `on_click_events` parameter of each button on click. Figure 26 shows a diagram of the overall system.

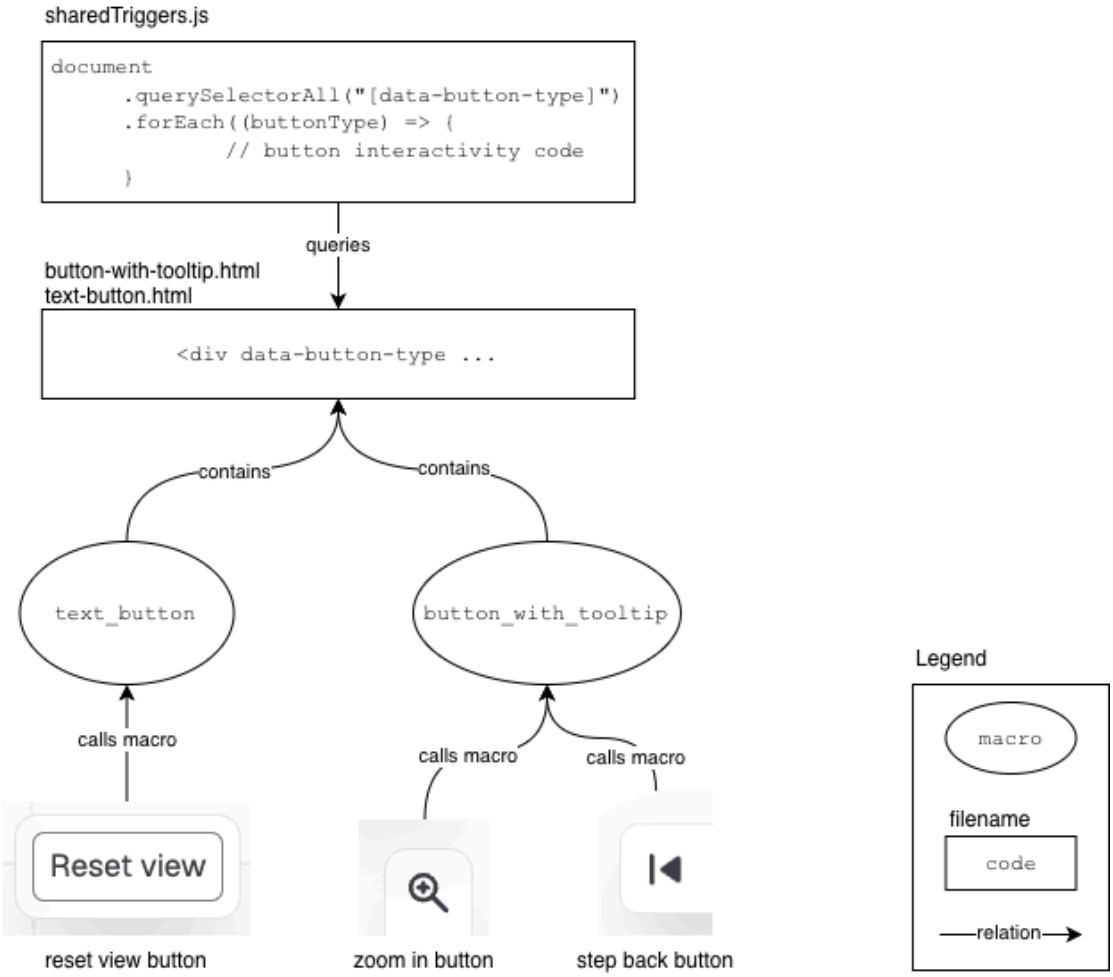


Figure 26

Diagram showing show event listeners are registered for buttons (`text_button` and `button_with_tooltip` macros). The button macros contain the data attribute `data-button-type`, which a JavaScript script queries and registers the events which are passed in as parameters to the macros. Then, each registered event is automatically fired when the button is clicked.

To respond to published events, controls use small UI scripts to subscribe to these events and update the page to reflect the new state. For example, the play/pause, theme, and full-screen buttons swap their icon and tooltip text,

overlays and side panels add or remove Tailwind classes to fade in or expand, and the step buttons' tooltips update to show the current step size. An example is shown in Listing 5.

Listing 5

In `simulation-playback-controls.js`, we subscribe to events published in response to UI interactions (e.g. buttons clicked, or inputs changed) to respond to these events. Here, when the simulation speed adjustment controls are interacted with, we ensure that the step forward/back buttons' tooltips are updated accordingly to reflect the new state.

```
bus.subscribe(EVENTS.SIM.ADJUST_SPEED, updateStepButtonTooltips);
bus.subscribe(EVENTS.SIM.ADJUST_SPEED_UNIT, updateStepButtonTooltips);
bus.subscribe(EVENTS.SIM.INCREASE_SPEED, updateStepButtonTooltips);
bus.subscribe(EVENTS.SIM.DECREASE_SPEED, updateStepButtonTooltips);
```

A diagram of the entire process is shown in Figure 27.

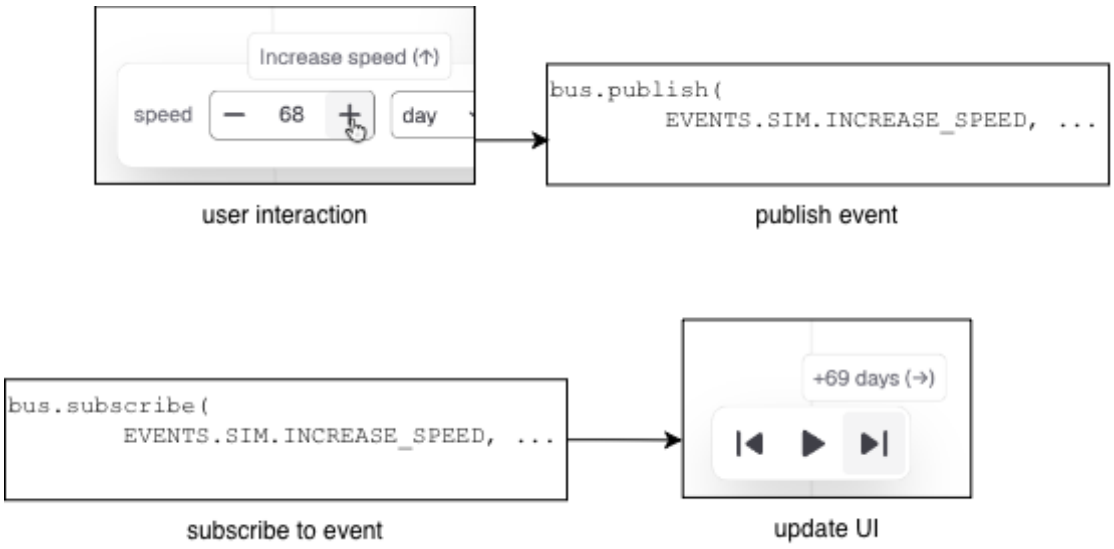


Figure 27

A diagram of the implementation of interactivity for controls using the event system. An event is published as a result of a user interaction (see Figure 26 for more detail). These events are then subscribed to so that interactions can be handled, such as updating a tooltip as shown.

Application-wide settings are applied by setting a single attribute or class on the root `html` element, such as `data-theme` for dark mode, `data-text-size` for the

larger text option, and `font-accessible` for the OpenDyslexic font, which the style sheet then uses to restyle the entire interface at once. Because the web server also renders each page with the user's saved settings already applied, the interface appears in the correct state before any JavaScript runs, and the scripts only take over for changes made after the page loads. This is further discussed in Section 7.3.4 (Applying state on page load).

The implementation of interactivity through the renderer and the backend is further discussed in Section 7.2 (Simulation renderer) and Section 7.4 (Event system).

7.1.5. Navigation flashes

Ethan Gee

An unforeseen problem that we encountered during development was that the application window would sometimes flash white when navigating between the simulation pages in dark mode. We first suspected that the simulation background was being applied late, as the simulation page has to set up a scene and might be slow to load. We then suspected a GPU issue, as the flash was more noticeable on some devices. However, neither turned out to be the source of the flash. So, to debug this issue properly, we recorded a video of navigating between simulation pages and stepped through it frame by frame. This showed that the flash occurred while the old simulation page was unloading, which takes a noticeable amount as the entire scene and its components have to be disposed of. During this time, the dark theme had been unloaded from some DOM elements but not others, including the simulation background, which was now white (see Figure 28).

We could not find a robust way to change the unload behaviour so that the theme is kept until the page has fully unloaded, so we added a workaround instead. In the Electron main process, we listen for the `will-navigate` event on the main window. When page navigation begins, the main process uses `executeJavaScript` to hide the simulation background in the page being navigated away from. It then waits for a promise to resolve after two consecutive `requestAnimationFrame` callbacks, allowing the change to be rendered before the page begins unloading. The user

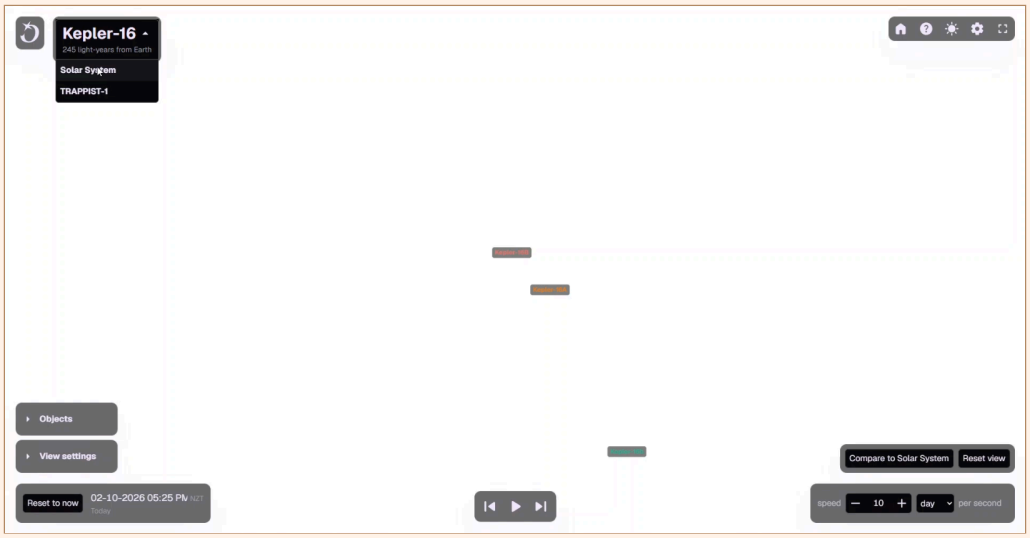


Figure 28

Frame captured while the old simulation page is unloading.

then sees the background of the application window, which always matches their theme, so the visible flash is completely removed and navigation appears seamless.

7.1.6. Help menu

Derek Chai

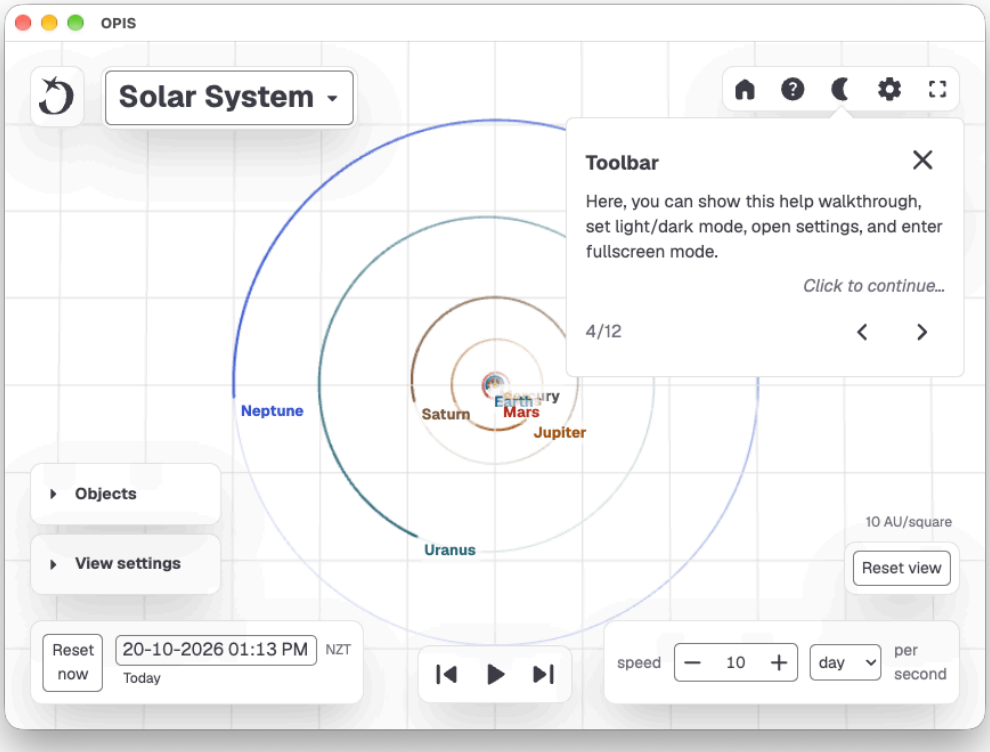


Figure 29

The help walkthrough of OPIS, showing a tooltip describing the toolbar.

Initially, we intended to implement the help menu as a simple screenshot annotated with descriptions of the functionality of each control. However, we realised that this approach, while relatively simple, would not be extensible. Additionally, it also has accessibility/UX implications, as users would need to constantly switch between having the menu open and referencing the actual simulation UI.

Ultimately, we decided the best approach was to integrate a walkthrough-style help menu, where a sequence of tooltips appear next to the control which they describe. This has the advantage that the context is immediately evident from the collocation of the tooltip and control, preventing constant context-switching. Additionally, this is a more extensible approach as adding a new help annotation requires only an additional Jinja macro call. The styling of the tooltips are also centralised and can easily be modified.

The positioning of the help menu tooltips is implemented using the Floating UI[12] JavaScript library. Floating UI automatically handles the placement of tooltips and overlays using 'middleware' which defines the tooltip placement behaviour, such as offsetting a tooltip from its target (`offset`), flipping the tooltip to the opposite side of the target if it would otherwise be shown off-screen (`flip`), or completely automatic handling of the tooltip placement (`autoPlacement`).

Using Floating UI, we can ensure that tooltips always appear entirely on-screen, shown in Figure 30, and that tooltips move as their targets move, shown in Figure 31.

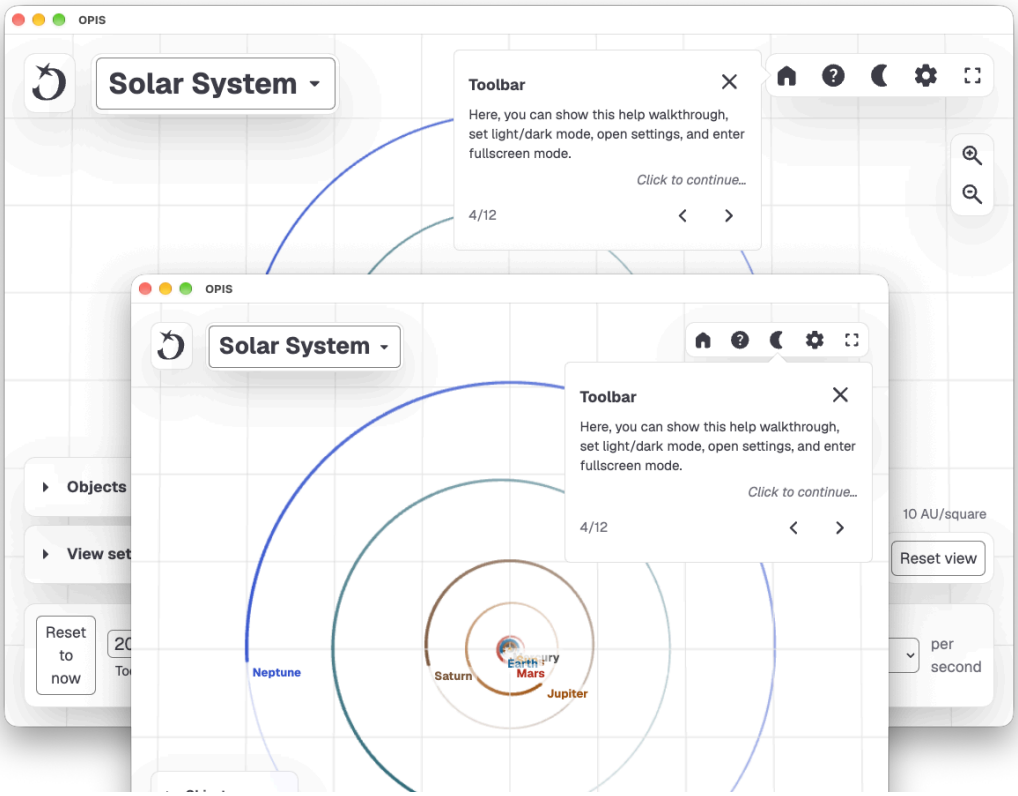


Figure 30

Floating UI automatically places tooltips so that they appear with the most space possible. Notice in the larger window, the tooltip is placed to the left of the target, while in the smaller window, the tooltip is placed beneath the target.

[12] https://floating-ui.com

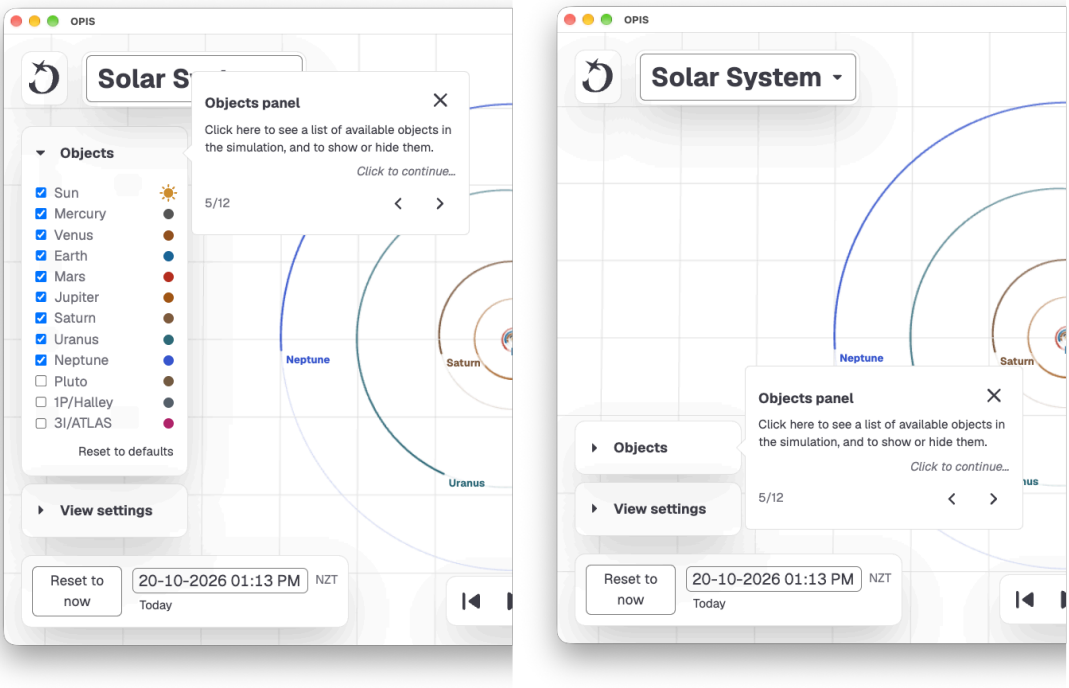


Figure 31

Floating UI ensures that the tooltip's position is always updated to be next to its target. Here, the tooltip moves as the Objects *panel is expanded.*

To handle user interactions and respond to them accordingly, the help walkthrough adds the following events:

```
export const EVENTS = {
  HELP_WALKTHROUGH: {
      NEXT: "help_walkthrough:next",
      PREVIOUS: "help_walkthrough:previous",
      SKIP_ALL: "help_walkthrough:skip_all",
  },
  ...
```

Certain user interactions, including clicking on the simulation canvas on `simulation.html`, or clicking outside a button or a container on `home.html`, will publish the `EVENTS.HELP_WALKTHROUGH.NEXT` event which a JavaScript file `helpWalkthrough.js` subscribes to to advance to the next step in the walkthrough. Similarly, other interactions will publish the other events, which the same file subscribes to and handles.

7.2. Simulation renderer

Jianing Li

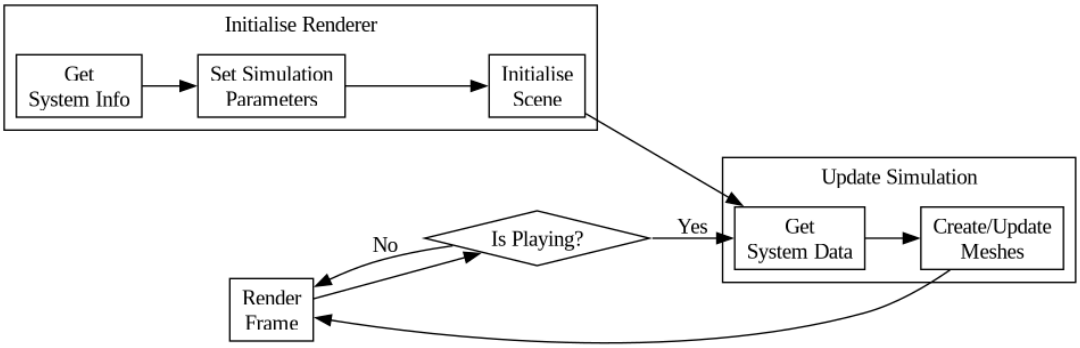


Figure 32

Simulation renderer flow chart

7.2.1. Rendering framework

The planetary system simulations in our application are rendered using Three.js, a 3D rendering library in JavaScript. High level abstractions are provided by Three.js for handling the scene, camera, controls, as well as managing geometries, materials, and meshes, significantly reducing the amount of low-level code that would otherwise be required. For example, the `OrbitControls` class allowed us to implement controls such as orbiting, zooming, and panning. Additionally, `RayCaster` enabled for the detection of object intersections on mouse clicks, and was used to implement user interactions such as selecting objects.

7.2.2. Rendering logic

A set of service functions in `simulationServices.js` is exposed to the simulation renderer to fetch data from the backend. To initialise the simulation, the `getSystemInfo` service function is called by the simulation renderer to retrieve information on the current system, which includes the orbital parameters:

- a (semi-major axis)
- e (eccentricity)
- Ω (longitude of the ascending node)
- i (inclination)
- ω (argument of periapsis)

System information also contains the habitable zone parameters, and the colours configured for each object, to set up the scene for the current system. Since this information is static, it is cached by the renderer for subsequent use.

The scene is then initialised using the `Scene` class, after which the habitable zone and reference grid are created and added to the scene. The `currentSystemGroup` and `solarSystemGroup` are also created and added to the scene using the `Group` class, with separate groups used so that attributes such as visibility can be controlled independently for the two systems. Since the `solarSystemGroup` is used by meshes for the Solar System comparison, it is not needed and therefore unused for the Solar System simulation.

After initialising the scene, the simulation is rendered in a render loop that repeatedly invokes `renderFrame` for each frame. While the simulation is playing, the `getMultipleSystemsData` service is called on each invocation to fetch object positions and orbital parameters at the current simulation time. This allows the renderer to update the meshes for objects and orbits in each frame during simulation playback. As only the transforms of meshes, i.e. positions and rotations, are updated in each frame based on the fetched data, rather than recreating the mesh objects repeatedly, there is relatively little computational overhead.

The simulation renderer also exposes a set of functions that allow the UI to control the simulation. This includes stepping through time, resetting the camera, and toggling 'Compare to Solar System'. For example, the `compareToSolarSystem` function is exposed to `simulationRendererController.js`, and displays the solar system when invoked. The renderer also exposes the functions controlling the view settings of the simulation, such as `setHabitableZoneVisibility` to display or hide the habitable zone mesh.

7.2.3. Objects and orbits

Using system data returned from the backend, we were able to position objects in our simulation relatively easily. As the returned position data are in the form of Cartesian coordinates in astronomical units (AU), we chose 1 AU as the world unit for the renderer, which allowed us to render the meshes directly using these values.

Displaying the orbital lines required performing calculations using the orbital parameters. The Cartesian coordinates of points along the orbit are

calculated with a (semi-major axis) and e (eccentricity). Using the polar equation for the ellipse and hyperbola, given by Equation 1, the distance between the object and the focal point (i.e. the barycentre) at angle θ is calculated, sampling 360 angles spaced roughly evenly.

$$r(\theta) = \frac{a(1-e^2)}{1+e\cos(\theta)} \tag{1}$$

Then, Equation 2 gives the Cartesian coordinates of the point on the orbital plane.

$$\begin{aligned} x(\theta) &= r\cos(\theta) \\ y(\theta) &= r\sin(\theta) \end{aligned} \tag{2}$$

Ω (longitude of the ascending node), i (inclination), and ω (argument of periapsis), in radians, are then used to calculate the rotation matrix R of the orbit, with $R = R_z(\Omega)R_x(i)R_z(\omega)$, where R_x, R_y, R_z are rotations around the x-, y-, and z- axes respectively. This rotation matrix is then applied to the orbit using the orbital line's `quaternion`, to ensure that the orbital plane is rotated correctly relative to the ecliptic plane†.

An unforeseen problem encountered after implementing orbital lines was that orbit inclinations varied significantly between systems. While the inclination of solar system orbits are generally close to 0 degrees (with the exception of Pluto and Halley's Comet), Kepler-16 and TRAPPIST-1 orbits are inclined at approximately 90 degrees to their ecliptic plane†. This meant that using the ecliptic plane† as the reference plane, defined as the xy-plane of the simulation, would result in inconsistent camera behaviour and discrepancies in the apparent orbit inclinations when comparing between systems. The up-direction of the camera would be roughly perpendicular to the orbital planes for the Solar System, whereas for Kepler-16 and Trappist-1, it would be parallel to their orbits. Therefore, to ensure for consistency between systems, we defined the reference plane of each system to be the plane perpendicular to the average normal vector of each orbital plane. Each system is rotated such that the average normal vector is aligned with the world up direction (z-axis).

7.2.4. Orbit line projections

Derek Chai

After the second client meeting, further outlined in Section 8.4 (Client testing 2 (13 October 2026)), we realised that, owing to the perspective (as opposed to orthographic) camera which OPIS' renderer uses, and the fact that Pluto's orbit is highly inclined, without a reference, it was difficult to see that Pluto's orbit comes closer than Neptune's for part of it.

To fix this issue, we had two potential solutions: switch to using an orthographic camera, or add lines projected from the orbit lines to the reference plane to provide a reference.

In the end, with advice from our client Larissa, we decided to solve this issue with projected lines. We chose this solution over switching to an orthographic camera because we wished to preserve the realism which the perspective camera provides.

Implementing the projection lines was relatively straightforward. As we already had an array of points which make up the orbit line, discussed in Section 7.2.3 (Objects and orbits), we only had to calculate the projected point of each point on the orbit line onto the reference plane, which is easily done with existing functions in Three.js.

Equation 3 shows the calculation of the projected point on the plane, $\boldsymbol{p}_{\mathbf{orbit}_{\text{plane}}}$. The line segment for each projection line is created from the two endpoints $\left[\boldsymbol{p}_{\mathbf{orbit}},\ \boldsymbol{p}_{\mathbf{orbit}_{\text{plane}}}\right]$.

$$\begin{aligned} \boldsymbol{p}_{\mathbf{orbit}_{\text{plane}}} &= \boldsymbol{p}_{\mathbf{orbit}} - \frac{\boldsymbol{p}_{\mathbf{orbit}} \cdot \hat{\boldsymbol{n}}}{\|\hat{\boldsymbol{n}}\|^2}\hat{\boldsymbol{n}} \qquad \text{for plane } \hat{\boldsymbol{n}} \cdot \boldsymbol{p} = 0 \\ &\text{where } \hat{\boldsymbol{n}} = (0\ \ 0\ \ 1)^{\mathrm{T}} \\ &\text{where } \boldsymbol{p} = \text{point on plane} \\ &\text{where } \boldsymbol{p}_{\mathbf{orbit}} = \text{point on orbit} \end{aligned} \tag{3}$$

The projection line is calculated for each point on the orbit and rendered as a child of the original orbit line, inheriting its properties including its colour and visibility.

A comparison of the presence of orbit line projections is shown in Figure 33.

This largely solves the confusing perspective at high zoom levels in determining whether Pluto's orbit is closer to the Sun than Neptune's. This is

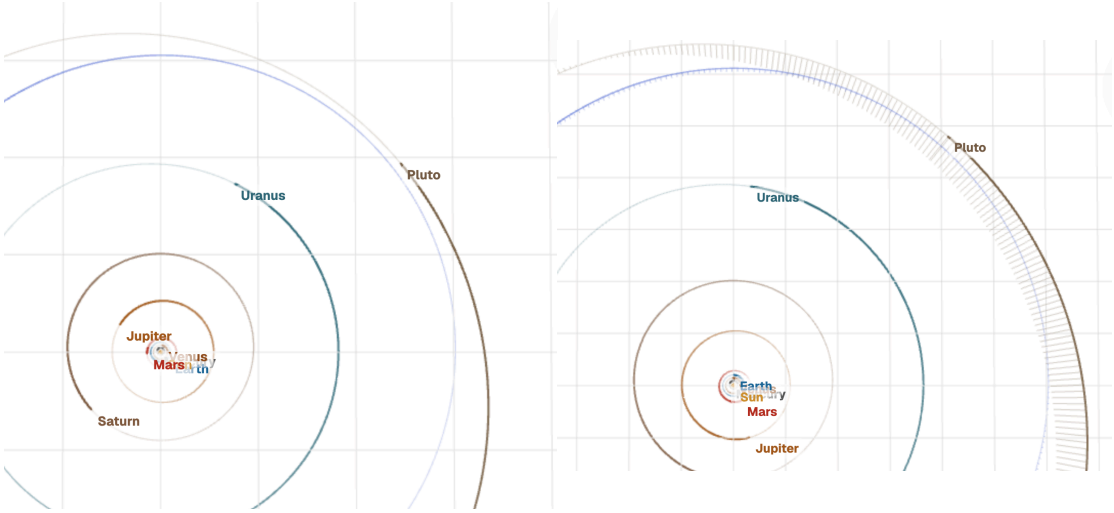


Figure 33

Left: Pluto and Neptune without orbit line projections. From this perspective, it appears that Pluto's orbit is always further from the Sun than Neptune's. Right: a similar view with orbit line projections. From the projected lines, it is clear that Pluto's orbit does come closer to the Sun than Neptune's.

made more obvious when the user rotates the camera to view the orbits from the side, as shown in Figure 34.

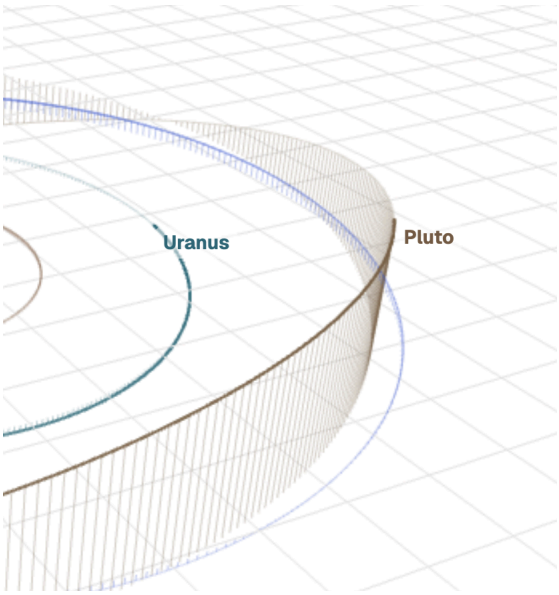


Figure 34

After rotating the camera, it is especially evident that Pluto's orbit comes closer to the Sun than Neptune's.

7.2.5. Habitable zone

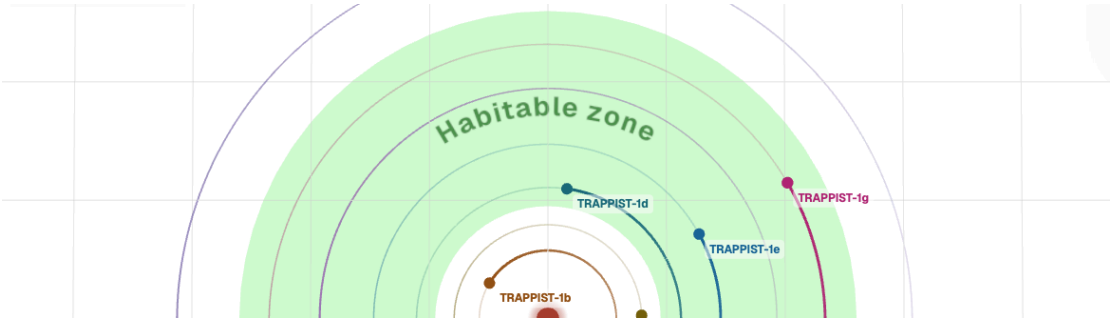


Figure 35

The habitable zone overlay in OPIS.

To render the habitable zone, the `start` and `end` values returned from the backend are used. These values represent the distance in AU from the barycentre where the habitable zone starts and ends, and are used directly with the `RingGeometry` provided by Three.js to display the habitable zone ring.

7.2.6. Tapered orbital lines

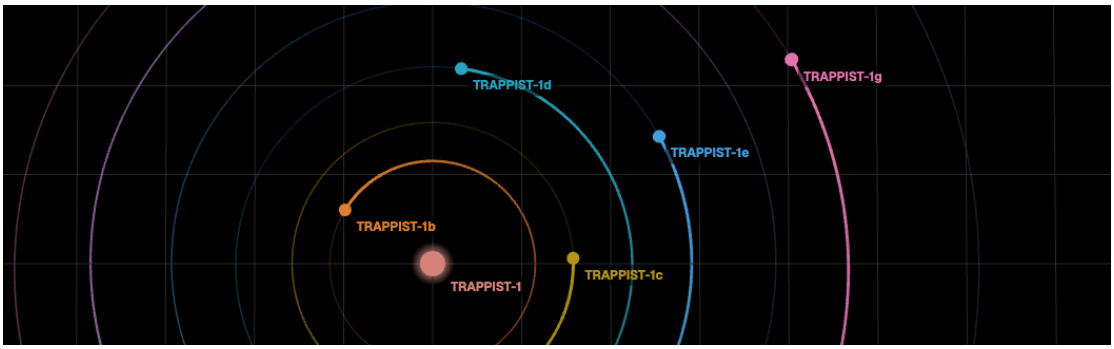


Figure 36

Tapered orbit lines in OPIS.

To improve visual aesthetics of orbits and to make each object's current position more easily identifiable to the user, tapering of orbital lines was implemented. The taper also indicates the direction that the object is travelling in even when the simulation is paused, with the line gradually becoming thinner and more faded behind the object.

We encountered several unforeseen challenges when implementing tapering orbital lines.

While `Line2` (the built-in Three.js line class) allowed us to implement orbital lines of a constant width and opacity, Three.js did not provide a way to easily vary these along the line, and doing this would require implementing a custom vertex shader. This would involve a considerable amount of work, and so we decided to search for an external library to simplify this implementation. However, we found that options for external libraries that met our requirements were quite limited.

After consideration, we decided to use the `three.meshline` library. A key limitation was that the library was no longer maintained. Given the limited number of options available, and since it was only used to implement a non-critical visual feature, we considered the maintenance risk to be acceptable.

Using the provided `MeshLineGeometry` and `MeshLineMaterial` classes, we defined functions to modulate the line width (Equation 5) and opacity (Equation 4) along the line (at proportion of the full line length d), and applied these to the lines' geometry and texture respectively.

$$\text{opacity}(d) = \max\{0.2, 1 - d\} \tag{4}$$

$$\text{width}(d) = \max\left\{0.5, \cos\left(\frac{\pi \cdot d}{2}\right)\right\} \tag{5}$$

However, there were additional complications encountered during implementation: a bug in the library resulted in uneven line widths depending on the application window's aspect ratio where the lateral sides of orbits have a different width from the vertical sides. Since the library was no longer maintained, we decided to vendor the library into our source code to implement a fix ourselves.

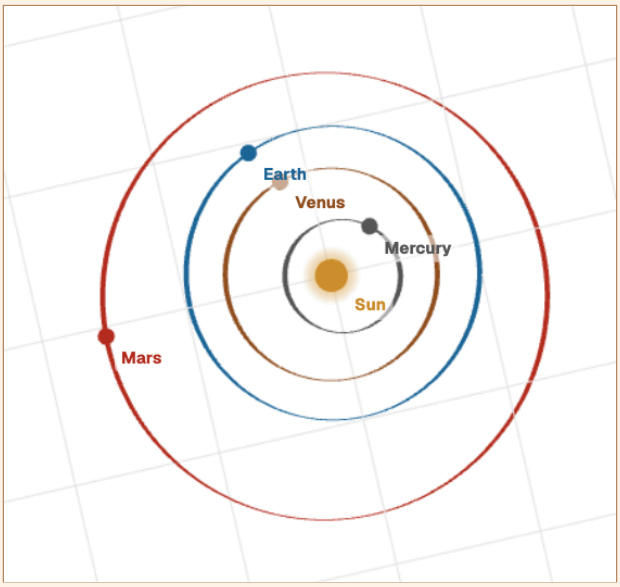


Figure 37

The visual bug encountered due to the issue in `three.meshline`. Notice that the lateral sides of the orbits appear thicker than the vertical sides. This occurred when the screen aspect ratio was very uneven.

7.2.7. Reference grid

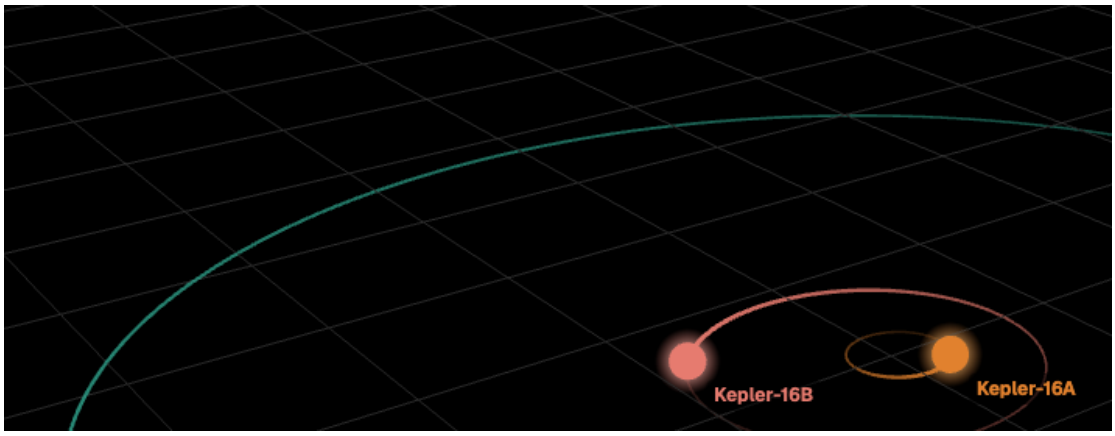


Figure 38

The reference grid in OPIS.

A reference grid aligned with the reference plane was implemented using the `GridHelper` class to help users orientate themselves spatially when navigating the simulation. It was important that the grid appeared infinite from all camera

positions, but as `GridHelper` produces a finite grid, this was much more challenging than initially anticipated. To achieve this, we limited camera panning by restricting the target within a fixed sphere around the barycentre, depending on the size of the planetary system, measured by the largest apoapsis of its orbits.

Scaling the grid squares depending on the camera position was implemented so that grid squares remain visually consistent and useful across different zoom levels. However, this presented additional complications as `GridHelper` does not provide an easy way of resizing grid squares, and instead required the entire grid to be scaled. This meant that the finite edges of the grid would become visible when it is shrunk which is undesirable. For example, an approach considered was to scale the grid depending on the camera's zoom, determined by the distance from the camera's position to its target. However, if the camera was panned away from the origin, the edges of the grid would become visible when zoomed in, and so a different approach was needed.

Eventually, we opted to scale the grid using the distance from the camera position to the projection of its target on the reference plane, with a minimum grid size set so that it does not shrink smaller than the size of the system. Since camera panning has been restricted depending on the system's size, the edges of the reference grid would not be visible to the camera regardless of camera panning or zoom. An added benefit for using this distance metric for scaling is that while the camera target is on the reference plane, the reference grid is only be scaled in response to camera zoom, and is unaffected by camera rotations.

7.2.8. Raycasting

Ethan Gee

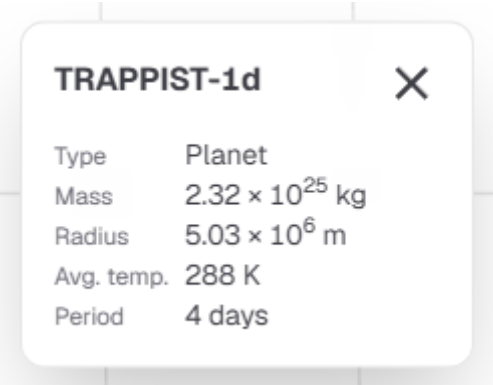


Figure 39

The object information panel for TRAPPIST-1d.

To display information about the objects in the current system without cluttering the user interface, a pop-up information panel was added. Clicking an object or its label opens a panel showing its details, as seen in Figure 39. Only one panel is shown at a time, so selecting another object replaces the current one. The panel can also be closed with the `X` button.

To detect clicks on objects and labels, `THREE.Raycaster` is used, which fires a ray from the camera and returns the scene objects it intersects. A custom `OBJECT_CLICK` event is fired for the closest intersected object, and the frontend handles this event by displaying the corresponding object information panel in the middle right of the user interface.

7.3. State management

Ethan Gee

7.3.1. State files

State is accessed from the application pages through two files, `settingsState.js` for persistent state and `simulationState.js` for session state. State is stored as a key-value pair, where the key names the state (for example, the application theme) and the value is what it is currently set to (e.g., `'dark'`). `settingsState.js` exposes the current settings state and a function `setSetting(key, value)`, which sets the value of the setting with the given key, updates the local copy of the state, and persists it so that it is retained between application runs. Similarly, `simulationState.js` exposes the current simulation state and a function

`setSimulationState(key, value)`, which sets the value of the simulation state with the given key and updates the local copy, but only retains the state for the current application run. In both files, the current state is held in a variable, which is initially populated by fetching the state from the main process each time a page is loaded.

7.3.2. State schemas

Both types of state have a schema, which is a JSON file including each key that the state can contain and its default value, as well as its options in the cases where its value is constrained. For example, the font size is limited to the options 'default' and 'larger'. The schemas are also used to initialise the state. Session state is populated in memory from its schema when the application starts, and the configuration file for persistent state is created from its schema if it is missing, such as when the application is launched for the first time.

7.3.3. Fetching and saving state

As the application's pages run in an Electron renderer process, they cannot directly invoke functions in the main process, so fetching and saving state is done through a preload script. A preload script is a script that Electron runs in the renderer process before a page loads, which can communicate with the main process and expose selected functionality to the pages. We use it to expose an API for settings state and an API for simulation state through `contextBridge.exposeInMainWorld`, which provides the pages with access to state stored in the main process. Each API exposes `get()` and `set()` functions. When `get()` is called by the application pages, the preload script uses Electron's `ipcRenderer` module to invoke a handler registered in the main process, which returns the corresponding state. For simulation state, the handler returns the values stored in memory in the main process. For settings state, the handler returns the values held in the store. The store is provided by the `electron-store` library, which persists its contents in a configuration file on disk. Saving state follows a similar process through `set()`. When a value is saved through `setSetting` or `setSimulationState`, it is passed through `set()` to a handler in the main process, which writes the state to memory (for simulation state) or to the store (for settings state).

7.3.4. Applying state on page load

One problem we encountered was how to display state in the user interface once it had been fetched from the main process.

Initially, each page fetched its state after it had loaded, and then updated the DOM using JavaScript to reflect that state. However, this led to a visually unpleasant flicker, as the page was first displayed without its state, before being updated to display it.

For example, the simulation date was not displayed until after the page had loaded. To avoid this, we used templating with Jinja so that the state is already present when the page is rendered. The main process listens for the `will-navigate` event, which fires when the user is about to navigate to a new page. Based on the URL being navigated to, state is added to the URL's search parameters. Settings state is added for every page, whereas simulation state is only added for a simulation page. When FastAPI receives the request for the page, it reads the search parameters and passes them into the context of the `TemplateResponse` method that renders the page. The state is therefore injected as the page is rendered, which avoids any flickering.

7.3.5. Resetting state

A user may want to reset the application to its default settings to start again from a clean slate. They can reset the view settings state (whether the habitable zone, orbits, reference grid, and labels are shown), the object visibility state (which objects in the current system are shown or hidden), or all state (including both session and persistent state).

The view settings and objects dropdowns each have a `Reset to defaults` button. When the reset button in the view settings dropdown is pressed, every event for a view setting (for example, `EVENTS.SIM.HABITABLE_ZONE_TOGGLE`, which toggles the habitable zone) is fired with that view setting's default value in its detail. These default values are fetched from `simulationState.js`, which initially fetches them from the main process via the preload script. Through the normal event flow, as detailed in Section 7.4 (Event system), the stored state and what

the user sees (for example, whether the habitable zone is displayed) are then updated accordingly. Similarly, when the reset button in the objects dropdown is pressed, `EVENTS.SIM.OBJECT_TOGGLE` is fired for each object with its default visibility in its detail, which updates the stored state and shows or hides the object. However, this approach does not scale well to resetting all state, as an event would have to be fired for every key in the state. Instead, current state is overwritten with the default state, then the current page is reloaded. Finally, the state is added to the current page's URL as search parameters, so the reloaded page is rendered with the default state, as described in Section 7.3.4 (Applying state on page load).

7.4. Event system

Ethan Gee

The names of the custom events that can be published are declared in `events.js`. Events are published and subscribed to through a single shared instance of the `EventBus` class in `eventBus.js`, which extends `EventTarget`. `EventTarget` is a built-in browser class that lets an object dispatch events and register listeners that are notified when those events are dispatched. `EventBus` provides two methods: `publish(name, detail)`, which publishes an event with the given `name` and the data in `detail`, and `subscribe(name, handler)`, which registers a handler function to be called whenever an event with the given name is received.

Triggers are implemented across many files in the `triggers` directory, such as `settingsTriggers.js` and `simulationTriggers.js`. A trigger registers a listener for a DOM event, such as a button being clicked, a key being pressed, or text being entered, and publishes a custom event when the DOM event fires. For example, when the simulation speed is increased with the `+` button, the trigger first clamps the new speed to its limit of 0 to 100 years per second, then publishes the `EVENTS.SIM.ADJUST_SPEED` event, with `detail` containing the system the speed was adjusted for and the new speed.

Controllers subscribe to the events relevant to their component, and invoke functions in that component when those events are received. For example, in `simulationController.js` the `EVENTS.SIM.ADJUST_SPEED` event is subscribed to, and when the controller receives the event, `setSimulationSpeed(system, speed)` is invoked in `simulationState.js`, which updates that system's simulation speed in the session state. Other controllers

can subscribe to the same event, so a single event can result in function calls across several components.

7.5. Electron

Elle Fleming

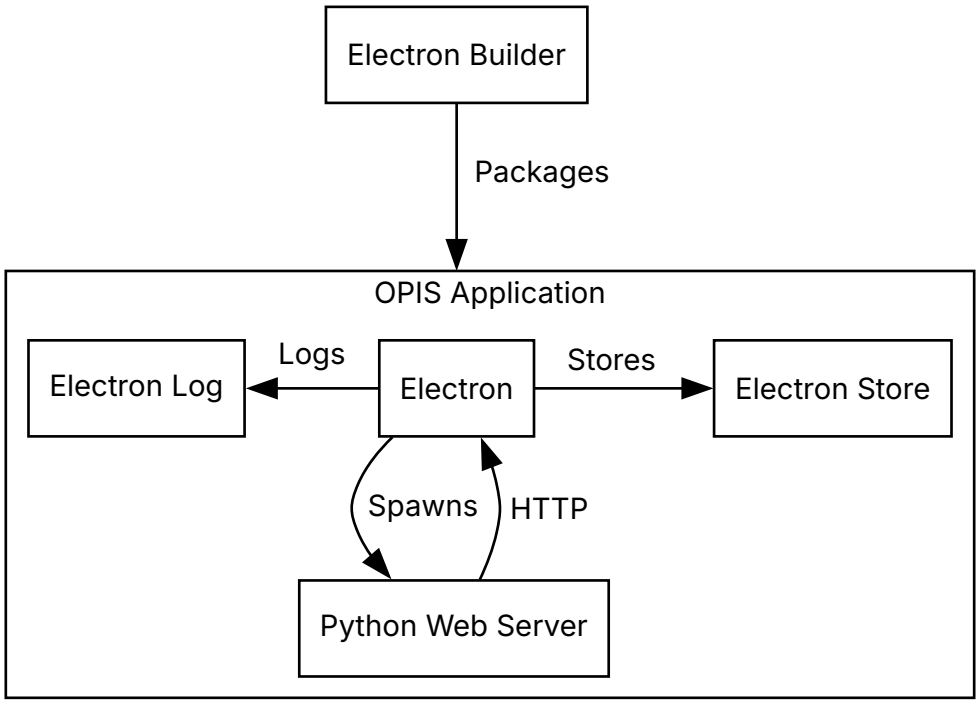


Figure 40

Electron app composition overview

The JavaScript desktop application framework Electron was used to wrap the Python web server as shown in Section 7.6 (Backend) and to package it into a single, easy-to-use desktop application. Electron is launched first and spawns the Python web server subprocess. This helped ensure our requirement of no performance issues as there is a local web server for each application preventing multiple requests and calculations needing to be served to multiple users simultaneously.

We used Electron Builder as our packager to produce the target executable, which produces an installer and a standalone executable. Electron also provides the benefit of cross-platform builds and development: this was not a requirement given to us, but was essential to our development process as members of the team developed across macOS, Windows, and Linux. Electron

also simplifies state management as discussed in Section 7.3 (State management). Jinja templating is used in the backend in order to react to the state given from Electron. We achieved logging, which is essential for long-term support, via Electron Log, which allows for simple automatic storage of log files.

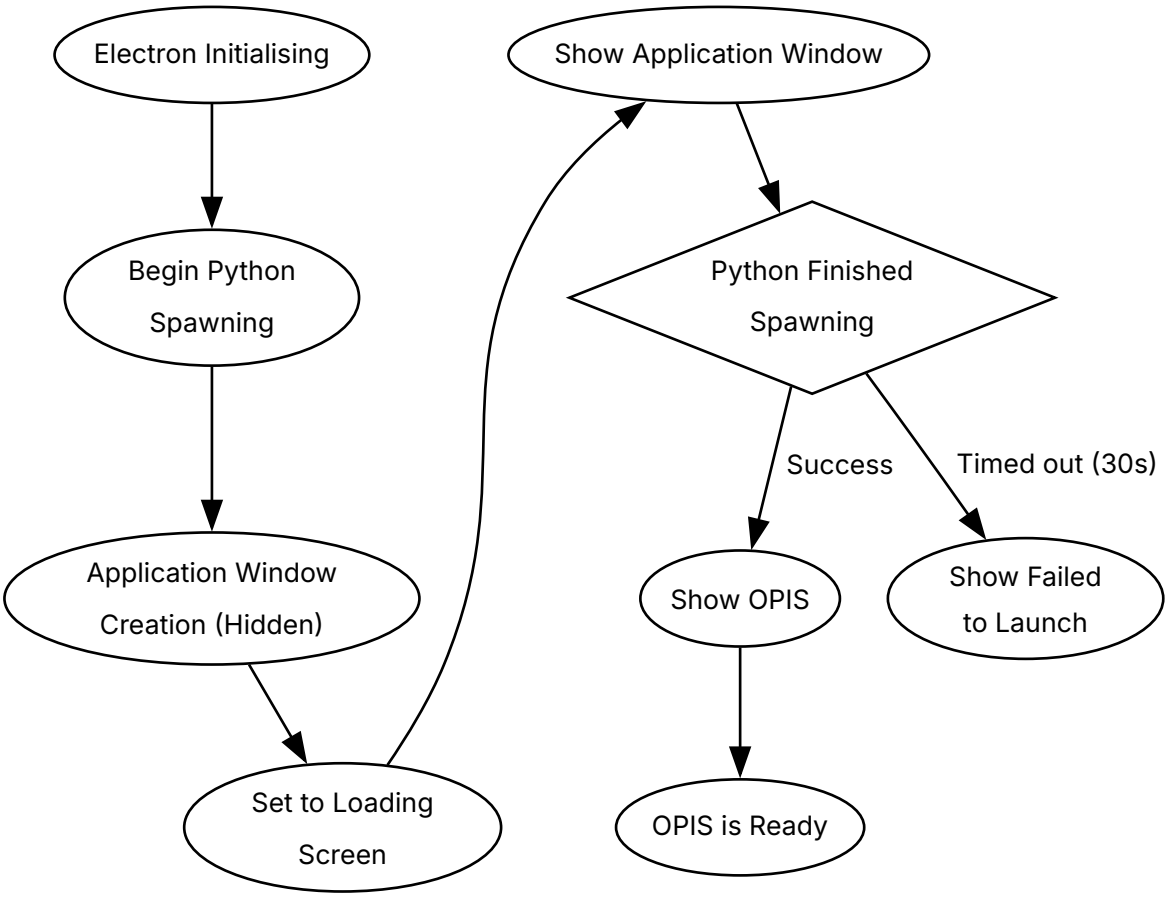


Figure 41

Electron application lifespan phases diagram

The implementation of Electron is kept entirely to the single file `index.js` since it is not used for the application's primary logic (unlike typical Electron applications) such as serving pages. Figure 41 shows the phases which the Electron application goes through:

1. Initialise everything required:
    - The user's application settings such as dark mode, dyslexic font, orbit lines style, etc.
    - `electron-logger` package and output format

- Application exit behaviour (ensuring Python exits correctly)

2. Spawn the Python subprocess, using a Promise, dependent on whether the application is packaged or development mode
3. Create the window with its required parameters, e.g. minimum size, logo, title, etc.
4. Show the loading screen while Python process launches
5. The Python promise then returns, having either succeeded and thus returned the address of the server or timed out (after 30 seconds)

We ran into many issues when attempting to create our build process due to OS-specific behaviour.

One of the issues was that Windows builds executables ending in `.exe`, while UNIX-type systems have no suffix. This was solved by checking which OS the application is running on and choosing the file path to the Python executable accordingly.

Another issue was, for macOS, the version of Electron we use is not functional on certain versions of Node.js, and so we found a working version and pinned the version OPIS uses to this version (Node.js version 24.22.1).

7.6. Backend

Elle Fleming

The backend was implemented using a combination of technologies in Python: FastAPI for serving routes, REBOUND for astronomy calculations and Jinja templating for dynamic UI pages. Python was chosen to meet the client's requirement as the teaching team is the most familiar with it. REBOUND is also already well known by our client and the teaching team.

7.6.1. FastAPI and Uvicorn

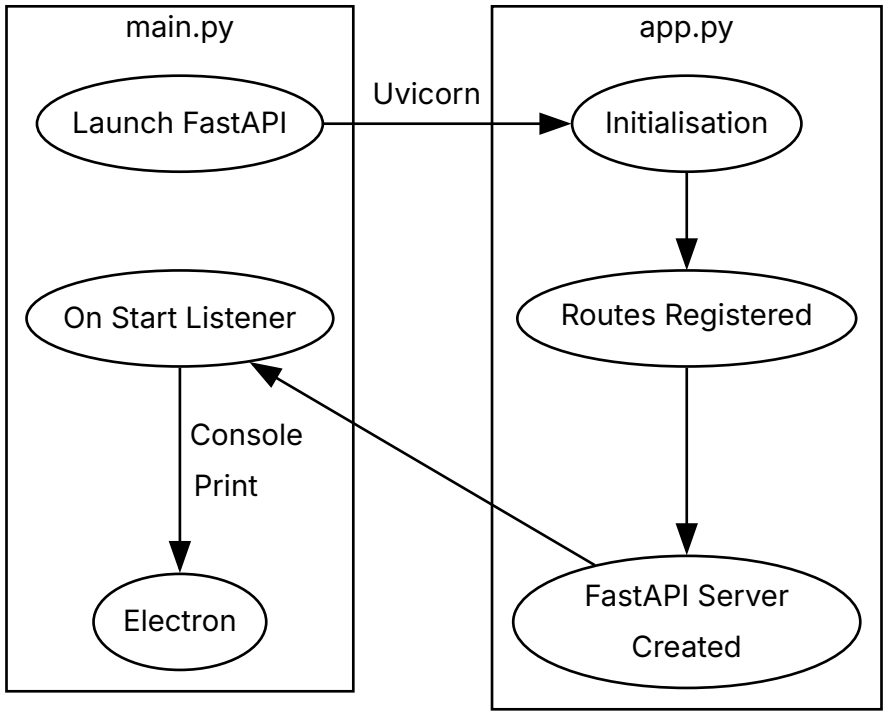


Figure 42

FastAPI launch state diagram

FastAPI was implemented in an unconventional way in OPIS as we needed dynamic port allocation and for the FastAPI server to print the IP address and port directly (e.g. `127.0.0.1:31784`) so that Electron could obtain the URL to the server. FastAPI does not natively provide any way to do this and so we used Uvicorn (the web server framework that FastAPI is built on) to achieve this. `main.py` launches the FastAPI server and registers an on-start listener function that can report the server's IP address and print it as seen in Figure 42.

Table 6 shows the five code-based and two static file-serving endpoints implemented in FastAPI.

Table 6

Descriptions of the backend endpoints implemented in FastAPI.

Endpoint	Explanation
`DELETE /current_integration`	This is the most complex endpoint and is discussed in depth in Section 7.6.3 (Maintaining interactivity) and detailed in Figure 44.
`GET /?settings +fullscreen`	This endpoint returns the HTML for the `home.html` page and is what users see when OPIS has launched. It uses Jinja templating based on the settings that are passed as query parameters and is initially given by Electron as described in Section 7.3 (State management).
`GET /simulation /system_name ?state +settings +fullscreen`	This endpoint returns the HTML for the `simulation.html` page and is set to show and get the correct simulation by Jinja templating. Query parameters are also passed into the Jinja template to use, for example, showing the systems in the dropdown.
`GET /system_info ?system_name`	This endpoint is used by the renderer to get information about a system such as the objects, habitable zone, and the reference frame for camera orientation. The renderer only requests this endpoint once after loading `GET /simulation` and the data is cached to stop expensive recalculations of the data.

Endpoint	Explanation
`GET /system_data ?system_names +t`	This endpoint calculates the given list of systems' data at the given time through REBOUND. Multiple systems can be given to this endpoint when comparing systems (i.e. compare to solar system). This endpoint is not guaranteed to return the complete data as it can be interrupted by the `DELETE /current_integration` endpoint.
`GET /src/*`	Statically serves all files in the `./src` directory, e.g. HTML and JavaScript files.
`GET /dist/*`	Statically serves all files in the `./dist` directory. Primarily used to serve the Tailwind-generated CSS file.

7.6.2. REBOUND and astronomical calculations

REBOUND is naturally stateful as it is designed to integrate (calculate the position of celestial objects) in a sequential manner.

When the user requests to reset the simulation to now after playing for a long period of time, since REBOUND is stateful, it has to integrate back the amount of time played. We designed a solution we call 'quick integration' which determines whether the time requested is closer to the current time (`sim.t`) or the initial simulation time (in `./systems/`): if it is, we reset. Figure 43 shows the implementation as a flow chart. This was necessary as REBOUND integrates from the timestamp last integrated to; e.g. if the simulation time $\texttt{sim.t} = 5$, then integrating to $\texttt{t} = 10$ only integrates from $\texttt{t} = 5$ to $\texttt{t} = 10$.

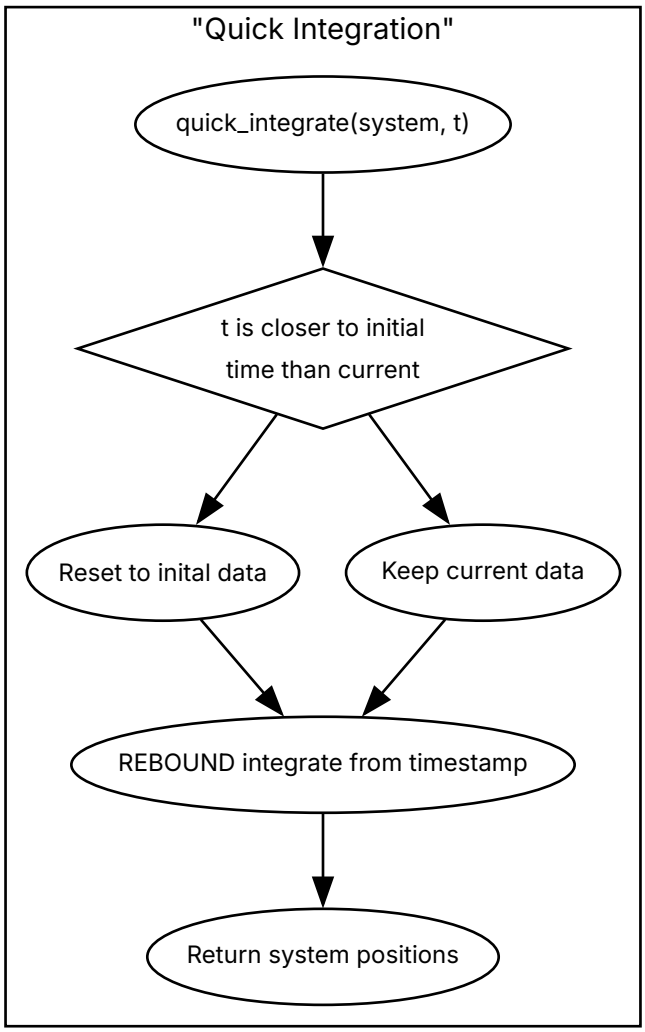


Figure 43

Quick integration flow chart

We also needed the orbits of the celestial bodies which was difficult as true orbits follow a spirograph-like shape. Thus we had two options to display orbits: a trajectory projection based system which calculates the position the object will be or has been and displays that as the orbit, or approximation ignoring external forces via osculating orbits. Where osculating orbits are the orbit at a moment in time ignoring all other factors. We chose the latter at the expense of a loss in accuracy (leading to unrealistic wobbles in the orbit) as the former is too computationally expensive. Equation 6 shows how these orbits are calculated.

$$\begin{aligned}
\overbrace{\mu_{\text{mass}}}^{\text{Average mass}}(p) &:= \sum_{q \in P \setminus \{p\}} q_{\text{mass}} && \text{For element } p \text{ in orbital object set } P \\
\mu_{\text{position}}(p) &:= \sum_{q \in P \setminus \{p\}} \frac{q_{\text{mass}}}{\mu_{\text{mass}(p)}} \cdot \boldsymbol{q}_{\textbf{position}} && (6) \\
\mu_{\text{velocity}}(p) &:= \sum_{q \in P \setminus \{p\}} \frac{q_{\text{mass}}}{\mu_{\text{mass}(p)}} \cdot \boldsymbol{q}_{\textbf{velocity}}
\end{aligned}$$

The pseudo-particle† (false object used for later calculations) $C(p)$ is given by Equation 7:

$$C(p) := \left(\mu_{\text{mass}}(p), \mu_{\text{position}}(p), \mu_{\text{velocity}}(p)\right) \tag{7}$$

With the pseudo-particle†, we can calculate the approximate orbit for each object about the pseudo-particle† (for that specific object) via REBOUND. More detail on the orbit parameters $[a\ e\ i\ ...]$ can be found in Section 7.2.3 (Objects and orbits). This solution generally works but breaks in specific cases where there are strong N-body perturbations† on the orbital element which is the case with Halley's Comet.

Since REBOUND simulates positions accurately, we stored the timestamp alongside the initial state of the system which allowed us to show the systems' positions at real times.

7.6.3. Maintaining interactivity

We encountered certain unexpected complications related to OPIS' UX.

For the UX to be smooth, the user needs to be able to stop the simulation if it is taking a long time to complete an action by clicking a prioritised action such as resetting the simulation to now. Thus, we had to be able to preempt the simulation's integration.

We first tried using subprocesses, where we spun a process up every time a request for system data was made so that we could kill the process preemptively. This is where we hit our first issue: Windows processes take a while (approximately 0.5 seconds) to start whereas UNIX systems achieve this near-instantaneously. Since Windows is required, we had to switch tactics.

We then attempted using the built in `heartbeat` functionality of REBOUND, which would allow us to check if the simulation should stop on every integration time step. This is where we encountered the second issue: this feature was not working at all on any of the REBOUND versions we tried. We decided to investigate the REBOUND codebase to see why it was non-functional and discovered the bug that caused this. The internal member variable that stores the C function pointer to the heartbeat function was not correctly registered. Our solution was to create a fork of the repository, fix the bug ourselves as we could not rely on the maintainers to do so quick enough, and create a pull request (https://github.com/hannorein/rebound/pull/938) into the library to share our fix with everyone. Our bug fix was eventually merged into REBOUND (version 5.2.2) and we set our application to permanently use this version to prevent future issues.

Our third issue we discovered was that it was non-trivial to use `heartbeat` to stop the simulation preemptively. Since FastAPI is multi-threaded by default, any part of the code was liable to run simultaneously at any time and so there were many race conditions. We decided the best way for priority updates to take priority was implementing an endpoint called `DELETE /current_integration` which terminated the integration that is currently running using a series of mutually exclusive locks and asynchronous events to prevent the race conditions, as seen in

Figure 44

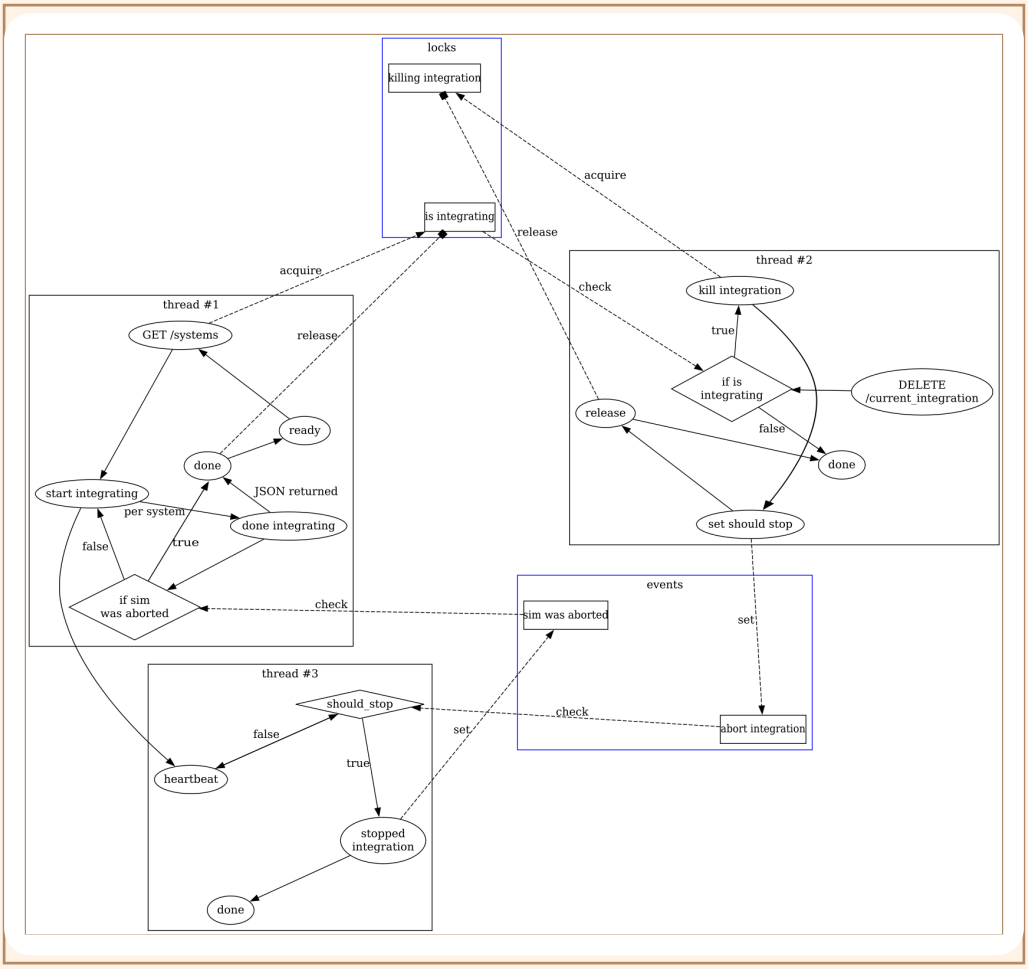


Figure 44

Backend `GET /systems` flow chart with priority events hitting the `DELETE /current_integration` endpoint

The renderer was made to make backend requests synchronously using promises and awaits to ensure queuing of requests does not occur as that causes looping of simulation playback.

7.6.4. Data sourced

All data we used for the backend in the `systems` directory was sourced only from peer-reviewed, scientifically-rigorous astronomy research papers or from the

[13] https://github.com/uoa-compsci399-s2-2026-anna/team-9-project/blob/main/References.md

NASA website. This data is cited in References.md[13] so that we, and maintainers of OPIS, could later reference the data if needed.

Masses and radii in literature are often given in multiples of masses/radii of the Sun (for stars) or of Jupiter (for planets). These needed to be converted to SI units in order to have a consistent set of units.

For mass of a star $M_{\text{star}}^{(\odot)}$ in multiples of solar masses and radius of a star $R_{\text{star}}^{(\odot)}$ in multiples of solar radii, Equation 8 shows the conversion from multiples of Solar masses/radii to SI units.

$$\begin{aligned} M_{\text{star}}^{(\text{kg})} &= M_{\text{star}}^{(\odot)} \cdot \overbrace{1.9891 \cdot 10^{30}\ \text{kg}}^{\text{mass of Sun}} \\ R_{\text{star}}^{(\text{m})} &= R_{\text{star}}^{(\odot)} \cdot \overbrace{6.96 \cdot 10^{8}\ \text{m}}^{\text{radius of Sun}} \end{aligned} \tag{8}$$

Likewise, for the mass of a planet $M_{\text{planet}}^{(\text{Jupiter})}$ in multiples of Jupiter masses and radius of a planet $R_{\text{planet}}^{(\text{Jupiter})}$ in multiples of Jupiter radii, Equation 9 shows the conversion from multiples of Jupiter masses/radii to SI units.

$$\begin{aligned} M_{\text{planet}}^{(\text{kg})} &= M_{\text{planet}}^{(\text{Jupiter})} \times \overbrace{1.898125 \cdot 10^{27}\ \text{kg}}^{\text{mass of Jupiter}} \\ R_{\text{planet}}^{(\text{m})} &= R_{\text{planet}}^{(\text{Jupiter})} \times \overbrace{6.9911 \cdot 10^{7}\ \text{m}}^{\text{radius of Jupiter}} \end{aligned} \tag{9}$$

7.7. Distribution and updates

Ethan Gee

7.7.1. CD pipeline

We use `electron-builder` to package OPIS, which produces a standalone executable and installer for each platform. Our CD pipeline is a GitHub Actions workflow that is triggered when a tag is pushed to our repository.

This workflow runs a build job on a Windows, macOS, and Linux runner. Each job checks out the tagged commit, installs the Node.js and Python dependencies, sets the application's version from the tag (e.g., `v1.0.2`), and packages the application with `electron-builder`. Once all three builds have finished, a final job publishes the installers as a single release on our repository's GitHub releases page[14] . The installer formats are `.exe` for Windows, `.dmg` for macOS, and `.deb`, `.rpm`, and `.AppImage` for Linux.

[14] https://github.com/uoa-compsci399-s2-2026-anna/team-9-project/releases

7.7.2. Auto-updater

We implemented the auto-updater using the `electron-updater` library, which is configured in `electron-builder.yml` to check our GitHub repository for new releases. When our packaged application is launched, `electron-updater` queries the GitHub releases API for the latest release of OPIS and checks whether it is newer than the installed version. If it is, a message box is displayed to inform the user that a new version is available and asks whether they would like to download it. If the user confirms, the new version is downloaded, and a second message box asks whether to restart OPIS now or later. Choosing to restart now restarts OPIS and installs the downloaded version, while choosing later installs it when the user next closes OPIS.

Our auto-updater cannot install new versions of OPIS on macOS. It can still detect that a new version is available, but automatic updates on macOS require our application to be signed with an Apple Developer ID[15] , which costs US$99 per year. We didn't consider this cost justifiable, as OPIS is primarily intended for the lab computers, which run Windows. As a result, on macOS, if a new version of OPIS is available, a message box is displayed to ask the user whether they would like to open our releases page, where they can download the new version of OPIS themselves.

[15] https://developer.apple.com/programs

8. Results and evaluation

To test OPIS, we held multiple testing sessions in order to test changes and gain feedback in an iterative manner. These testing sessions allowed us to gain insight into what worked well with OPIS, and what areas still needed improvement, especially in relation to the ASTRO 100 lab worksheet. We held a total of three formal user testing sessions across the development of OPIS, in addition to continuous informal testing with friends and peers.

8.1. Client testing 1 (25 September 2026) *Derek Chai*

8.1.1. Test plan

To test OPIS with users specific to ASTRO 100, we organised a user testing session with the ASTRO 100 tutors. This enabled us to gather immediate feedback on the utility of OPIS with the people who will be using the application in the future themselves. This test session lasted around 30 minutes.

We instructed the tutors to try to answer the questions in the lab worksheet used in ASTRO 100 using OPIS. We did not instruct the tutors in any way on how to use OPIS, rather allowing them to discover for themselves how to use the tool.

8.1.2. Results

Generally, the feedback we obtained from the tutors was very positive.

- All of the functionality required to answer the lab questions were included in OPIS.
- No testers struggled to such an extent that external assistance was required.
- Most of OPIS' features were very intuitive to testers (except the features mentioned in the following).

Key areas of improvement, where we noticed the tutors struggling in, or where OPIS could have been made more intuitive, were in:

- Ensuring that there is a distinctive differentiation between planets and stars in the simulation

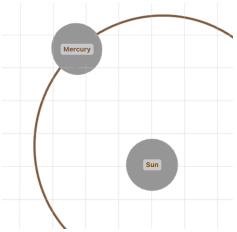
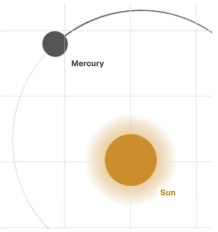
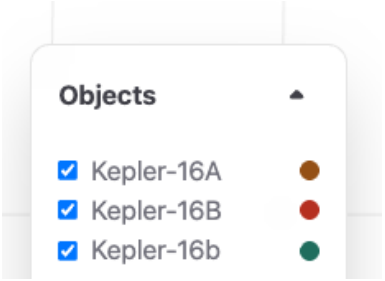
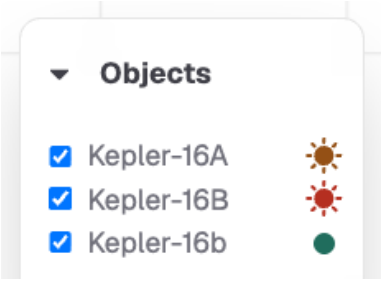
    - We found that it was not always intuitive, especially for Kepler-16, to tell apart planets from stars.
    - This is important because there is a question in the provided lab sheet which asks about how Kepler-16 is different to the Solar System (it is a binary star system)
- Making the date picker more obvious.
    - We found that the users largely did not interact with the date picker.
    - This may have partially been due to a lack of a specific requirement to use it in answering the lab worksheet.
    - However, at the same time, the existing date picker was not distinctive and did not suggest that it was a control which users can interact with.
- Making the habitable zone more obvious.
    - It was not immediately obvious what the habitable zone overlay was supposed to be.

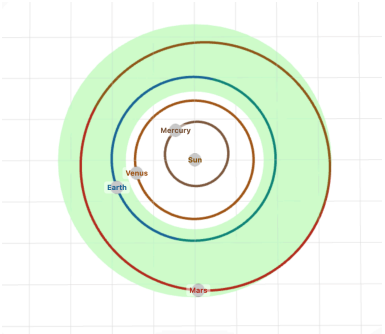
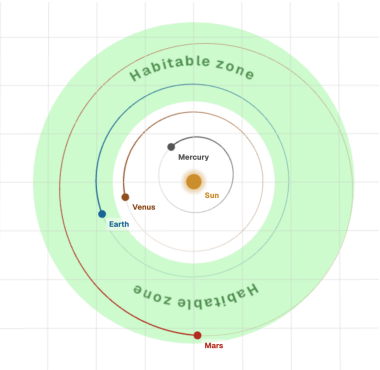
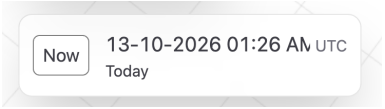
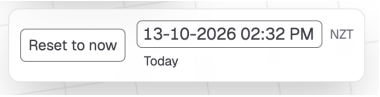
8.1.3. Actions

Table 7 outlines the actions we made as a result of the feedback from this round of testing.

Table 7

Actions taken as a result of the first round of client feedback.

Issue	Change	Before	After
No distinction between planets and stars	• Add glow around stars • Make stars appear larger than planets • Offset labels from objects so the full object size is easily seen		
No distinction between planets and stars	• Add a unique marker in the Objects list in the sidebar for stars • Add a field in the Object information panel describing the type of the object (star, planet, etc.)		

Issue	Change	Before	After
Habitable zone not obvious	• Add a label to the habitable zone • Make the habitable zone hidden by default, so users have to check the ‘Habitable zone’ checkbox to enable it.		
Date picker not obvious	• Add styling to the date picker to make it consistent with all other interactive buttons		

8.2. Peer testing (25 September 2026) *Derek Chai*

8.2.1. Test plan

Directly after the testing session with the ASTRO 100 tutors, we conducted a testing session with students from the COMPSCI 399 course cohort.

Similar to the first testing session, we instructed users to answer the questions in the lab worksheet using OPIS, with no guidance from us.

8.2.2. Results

Our results closely aligned those found in the first testing session. Again, the primary concern was a lack of differentiation between planets and stars.

8.2.3. Actions

The actions resulting from this testing session were combined with those in the first client testing session, shown in Table 7.

8.3. User testing

In addition to testing with ASTRO 100 tutors and with the COMPSCI 399 cohort, we also wished to test with ASTRO 100 students themselves to see how they would find using OPIS.

Unfortunately, after consultation with our client, we were unable to schedule a testing session for this purpose due to the tight timeline of the course.

Regardless, the testing session with the ASTRO 100 tutors was very useful and gave us insight into where OPIS still needed improving.

8.4. Client testing 2 (13 October 2026) *Derek Chai*

8.4.1. Test plan

Similar to the first client testing session, we intended to give testers the lab worksheet and instruct them to answer the questions using OPIS without external guidance.

[16] https://profiles.auckland.ac.nz/larissa-markwardt

This test was conducted with our client, Larissa Markwardt[16], and our course coordinator, Anna Trofimova[17].

8.4.2. Results

We found that all of the issues uncovered in the first client testing session had been addressed by our actions (outlined in Table 7). Namely:

- The distinction between planets and stars was very clear.
  - There was no confusion between what was a planet and what was a star.
- The habitable zone overlay was intuitive and obvious.
  - There was no confusion as to what the overlay represented.
- The date picker was much more obvious.
  - In the first testing session, no testers interacted with the date picker.
  - In this testing session, the date picker was interacted with to select a specific point in time to observe the approach of ATLAS.

However, this testing session also revealed some new issues:

- It is difficult to see that Pluto's orbit comes before Neptune's during part of its orbit.
  - This is especially important because there is a question in the lab worksheet which specifically relates to this:

> *Is Pluto always the most distant object from the Sun in [the Solar System]?*

- All orbits (including that of Halley's Comet, Pluto, and ATLAS) were being shown in the *Compare to Solar System* overlay.
- Certain visual bugs relating to the habitable zone overlay were discovered.
- There was no clear connection between the step forward/back step size and the simulation speed.
  - When the user clicks on the step forward/back button, the simulation steps forward/back by '1 second' of the current simulation speed.
    - For example, if the current simulation speed is 10 days per second, the step forward/back size is 10 days (i.e. 1 second of the speed).
  - However, this was not clearly communicated to the user.

[17] https://profiles.auckland.ac.nz/anna-trofimova

- It was frustrating having the simulation zoom out when an object was removed from the simulation.
- All planets have the same-sized markers, and all stars have the same-sized markers.
    - This is somewhat unrealistic as in real systems, certain planets/stars are larger than others.

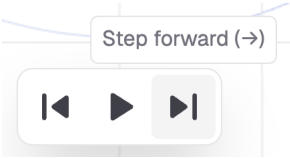
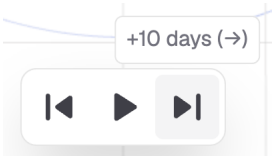
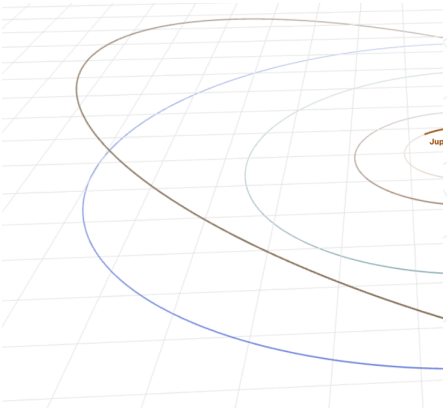
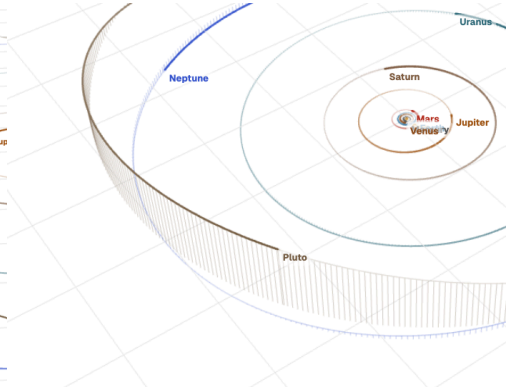
8.4.3. Actions

Table 8 shows some of the actions taken from observations during the second round of client testing. In addition to these new features, we also fixed the visual bugs which were uncovered during this testing session.

We were unable to implement the feature where objects' markers are rendered proportional/relative to their true size. With scales of different planetary systems varying so much, it was difficult to come up with a one-size-fits-all scale factor or function to convert the object's true size to its apparent rendered size.

Table 8

Actions taken as a result of the second round of client feedback.

Issue	Change	Before	After
No connection between step forward/back size and simulation speed	• Change button tool-tip to show step size		
Simulation zooms out when hiding an object; simulation always zooms out to maximum zoom level	• Do not zoom in/out when hiding an object • When showing an object, zoom in/out as required to fit the object's orbit		
Hard to see that Pluto's orbit comes closer than that of Neptune for part of its orbit	• Project lines from the orbit line to the reference plane		

8.5. Project goals evaluation

Derek Chai

Overall, we believe that OPIS achieved the vast majority of the project goals as outlined in Section 4.1 (Aims and objectives) and Section 6.1 (User requirements).

The main area in which we did not manage to achieve all project goals was in our accessibility goals, as outlined in Table 12.

Table 9, Table 10, Table 11, and Table 12 outline each user requirement as defined in Section 6.1 (User requirements), whether the requirement was met, and, if so, how it was met.

Table 9

Performance user requirements and whether they were successfully met in the final build of OPIS.

Performance user requirement	Met
Load application on lab computers used in lab within 5000 ms from launch	Yes: launches in at most 3870 ms[18] on lab computers
Load simulations within 1000 ms of user clicking relevant button	Yes: loads in at most 410 ms[19] on lab computers
Do not crash when used under normal conditions[20] in lab	Yes: no crashes experienced during user testing
Do not lag noticeably or stutter for longer than 100 ms after any user interaction under normal conditions	Yes: no lag experienced after user interaction under normal conditions Lag experienced if TRAPPIST-1 is played at an excessive speed (outside of normal conditions)
Run simulation rendering consistently at frame rate of at least 30 frames per second on lab computers under normal conditions	Yes: runs at 60 frames per second on lab computers
Support up to 80 concurrent users without noticeable performance impact	Yes: OPIS runs locally and thus is unaffected by concurrent users

[18] Slowest of 10 test launches
[19] Slowest of 10 test loads
[20] The conditions under which OPIS is expected to be used in answering the specific questions provided in the lab worksheet

Table 10

Functionality user requirements and whether they were successfully met in the final build of OPIS.

Functionality user requirement	Met
Observe accurate movement and orbits of objects within the Solar System, Kepler-16, and TRAPPIST-1	Yes: OPIS uses REBOUND to simulate object movement
Compare directly orbits of included exoplanetary systems[21] to those of the Solar System	Yes: *Compare to Solar System* feature
Overlay each planetary system's habitable zone over its orbits	Yes: *Habitable zone* overlay (in *View settings* side panel)
Play and pause simulation playback	Yes: play/pause button, or pressing space bar
Adjust simulation playback speed	Yes: adjustable from 0–100 years per second
Select an arbitrary date/time and have OPIS show positions of objects at that timestamp	Yes: calendar picker feature allows selecting any arbitrary date to view the simulation at that point in time
Rotate and pan around systems in three dimensions	Yes: left-click to rotate/orbit, right-click / shift + left-click to pan
Select simulated objects to obtain information about their masses, radii, object type, and mean temperature	Yes: object information panel displayed with facts (radii, masses, etc.) about object when object is clicked
Do not contain any unnecessary features—i.e. any features which are not directly involved in answering worksheet questions, or which cannot be used to answer such questions	Yes: the vast majority of OPIS' features are directly related/used within the worksheet. There exist some small additional features, but these do not negatively impact the ease of use of the application in any way
Simulate all planets and stars of the Solar System, Kepler-16, and TRAPPIST-1	Yes: OPIS includes all planets and stars of the Solar System, TRAPPIST-1, and Kepler-16

[21] Kepler-16 and TRAPPIST-1.

Functionality user requirement	Met
Simulate within the Solar System Halley's Comet and comet 3I/ATLAS in addition to all planets and stars	Yes: OPIS includes Halley's Comet, Pluto, and ATLAS in the Solar System simulation
Show realistic orbits, object positions, and distances to level of accuracy equal to or better than existing tools[22]	Yes: OPIS uses REBOUND as an N-body integrator to calculate object positions, which is accurate to machine precision [6]

Table 11

Deployment and packaging user requirements and whether they were successfully met in the final build of OPIS.

Deployment and packaging user requirement	Met
Be installable on lab computers (i.e. on Windows devices)	Yes: OPIS easily installable on Windows devices with provided exe executable. It is also supported on macOS and Linux
Provide all required functionality and user requirements in one single executable	Yes: OPIS is packaged into one single executable

Table 12

Accessibility user requirements and whether they were successfully met in the final build of OPIS.

Accessibility user requirement	Met
Be accessible to users with colour vision deficiencies	Yes: no features in OPIS are solely distinguished by colour
Be accessible to users with dyslexia or reading-related difficulties	Yes: OPIS features an option to use the OpenDyslexic font
Be useable with only the keyboard	No: OPIS is not entirely navigable with keyboard only
Be responsive to all screen sizes	Partially: OPIS is responsive to screen size changes, but we set a minimum window size for the application. Additionally, responsivity at small screen sizes is only implemented by zooming out the UI at these sizes

[22] i.e. Eyes on Exoplanets and Orbit Viewer.

8.6. Overall evaluation

Derek Chai

Overall, we believe that OPIS was a success. Table 13 outlines a critical evaluation of how well OPIS achieved each of the aims and objectives we outlined in Section 4.1 (Aims and objectives).

In combination with the evaluation against the user requirements shown previously in Table 9, Table 10, Table 11, and Table 12, we can see that OPIS has achieved the vast majority of its initial aims and objectives, and user requirements.

Of all our original aims and objectives, the one which was least met was that of modifiability. While we aimed to write self-documenting code and maintain clear file structures and interactions between different parts of the system, mainly owing to time constraints, we were unable to implement as much as we aimed for in terms of documentation and automated testing.

Table 13

A critical evaluation of how well OPIS achieved each of the aims and objectives described in Section 4.1 (Aims and objectives).

Objective	Description	Evaluation
Course specific	OPIS should provide all of the functionality necessary to complete the ASTRO 100 lab sheet, without any unrelated features which could otherwise distract students from the lab	+ OPIS features all functionality required to complete the ASTRO 100 lab sheet + OPIS' features and included systems are specifically tailored to the requirements of ASTRO 100 + OPIS has been developed alongside feedback from ASTRO 100 stakeholders, including tutors and lecturers

Objective	Description	Evaluation
Intuitive	Students should be able to find and use the functionality needed for each task on the lab sheet, with at most minimal technical assistance from teaching assistants	+ OPIS' UI has been specifically designed to be as intuitive and user-friendly as possible + User testing for OPIS was conducted under no external guidance for users on how to use OPIS. From test observations, all users were able to find and use the features required without assistance + OPIS features a help walk through to assist students in how to use the program should they still have difficulties
Scientifically accurate	The information and simulations that OPIS presents should be accurate	+ OPIS uses REBOUND as an N-body integrator, which is accurate to 'machine precision' [6] + The data used for calculations in OPIS is sourced from reputable sources including NASA and peer-reviewed literature + OPIS has been developed in consultation with domain expert Larissa Markwardt – Due to the nature of astronomy, there still remain inaccuracies, and orbits are not exact – Object sizes in the simulation are to to scale to orbit distances. This is due to practical reasons as the order of magnitude of both distances are vastly different
Reliable	OPIS should work consistently during the lab, performing well regardless of how many students are using it at once and should run on any laptop or desktop computer	+ OPIS does not rely on external servers or resources, and as such is unaffected by concurrent users + During user testing, OPIS never crashed + OPIS runs at 60 frames per second on lab computers

Objective	Description	Evaluation
Modifiable	The ASTRO 100 lab may change in the future, for example to include another exoplanetary system or different asteroids or comets; the ASTRO 100 teaching team should therefore be able to make such modifications without prior familiarity with the codebase, so that OPIS can continue to provide value as the course evolves	+ OPIS has been designed to easily allow for the addition of exoplanetary systems through a configurable `config.json` file – OPIS' developer documentation is largely incomplete – Parts of OPIS' file structure are messy and poorly-organised – Some technical knowledge is still required to maintain OPIS, including familiarity with the technologies used, as well as programming knowledge – Automated testing was not able to be included – Error handling was not comprehensively implemented

Outside of development-related issues, we also believe that the teamwork aspect of OPIS' development also went well.

Our team managed to almost completely adhere to all the conditions and agreements outlined in our final team contract (after it was first revised during the team contract revision assignment). Our team meetings were also very productive, with complete attendance of every meeting. Finally, our team organisation and project management approach to development also worked well and allowed us to rapidly design, prototype, and implement features.

9. Future work

9.1. Arbitrary planetary systems

Jianing Li

Currently, our application only supports the three planetary systems studied in ASTRO 100: the Solar System, Kepler-16, and TRAPPIST-1. In our current implementation, data is manually sourced from research papers to retrieve the parameters needed for these systems' simulations. However, this is time-consuming and would not scale to an arbitrary number of planetary systems. As such, an automated data pipeline to retrieve system data from a reliable source would need to be implemented. To avoid runtime dependencies on external APIs, this data would need to be directly bundled with the application so that OPIS continues to be fully functional offline.

The most promising API we found that provides data on a large collection of exoplanet systems was exoplanet.eu[23] . However, we found that this data was incomplete and some of the necessary parameters needed for initialising a simulation in REBOUND were missing. One solution to overcome this is to fall back to plausible values if some of the less important parameters are missing. For example, if the inclination i of the orbital plane was missing, a reasonable default fallback could be to set i to 0. However, parameters that describe the size and shape of the orbit, such as the semi-major axis a and eccentricity e, are essential for an orbit, and we would not be able to display an orbit without these. To ensure that these predefined fallback values are reasonable, they would be decided in consultation with domain experts.

9.2. Object following

Jianing Li

To let users observe objects more easily while the simulation is playing, a future feature would be to implement object following, which would allow users to lock the simulation camera's target onto an object. This is currently unimplemented as it was not a core requirement for OPIS.

To implement object following, the simulation renderer would need to keep track of the current position of the object being followed, and update the camera's target to the object's latest position in every frame, so that the camera

[23] https://exoplanet.eu

remains locked onto the object while the simulation is playing. There are several possible approaches to begin and stop following objects. A simple approach would be to follow objects when they are double-clicked. However, this has the downside that it is not clear that objects may be followed via double-clicking. Thus, an alternative approach would be to implement a button for following objects in the object information panel (see Figure 39), which is already implemented and accessible through clicking the object. This button would be used to toggle whether or not the object is being followed.

9.3. Proportional object sizes

Jianing Li

Non-stellar† objects in simulations are currently sized uniformly with each other, with stars being double the size. For example, Jupiter's radius is approximately 69 911 km compared to Earth's radius at 6 371 km, but in our current implementation, Earth and Jupiter appear the same size since they are both non-star objects. A future enhancement would therefore be to implement object sizes such that they are proportionally sized relative to each other. Data on object radii is already available within our current object configurations stored in JSON files within the `systems/` folder, and would be readily accessible by the simulation renderer.

A possible approach would be to display all objects as their true size relative to space. However, this has immediate downsides in that the vast distances between objects compared with their sizes would mean that most objects would not be visible at a reasonable zoom level. To resolve this, a scaling factor would need to be introduced, such that objects are sufficiently large to be visible, while still being sized proportionally to each other. Additionally, the scale disparity between objects would also need to be considered, as stars are orders of magnitude larger than non-star objects like planets. This would mean that a scaling factor that is sufficient for the smallest object to be visible may result in the largest object being overly large, and so a logarithmic scale would likely need to be used to address this. Finally, the size of systems can vary significantly, and so this would also need to be accounted for when implementing relative sizes, for it to be generalised across different planetary systems.

9.4. Automated tests

Jianing Li

Currently, there are no automated tests implemented in the codebase, due to time constraints and the complexity with introducing a comprehensive test suite. This would however be an important enhancement for the future, to verify that logic behaves as expected and prevent any regressions in behaviour as OPIS evolves over time.

9.4.1. Frontend and rendering tests

Unit tests and integration tests could be added for verifying frontend and rendering logic, using a JavaScript testing framework like Jest[24] . Unit tests would be used to test that isolated components function correctly, and would also cover utility functions such as verifying that the calculations used for rendering the simulation are implemented correctly.

Integration tests would verify that components behave as expected when interacting with other parts of the application. This is especially important for rendering logic because components are closely interconnected, and interactions in one component can affect the behaviour of several other components. For example, when 'Compare to Solar System' is pressed, an integration test would need to verify that the Solar System's object meshes and orbital lines are correctly displayed, and that the camera zoom is updated to fit the entire Solar System within the view. The UI would also need to be tested here, to verify that the button's text has correctly switched to 'Hide Solar System'.

9.4.2. Backend tests

Unit tests could be added to the backend using a Python test framework like pytest[25] . Calculation logic and utility functions such as `get_osculating_orbit` and `format_sim_date` on the backend could be tested. Testing would be relatively light however, as many of the calculations are handled by established libraries. Integrations are handled by REBOUND and time conversions are handled by Astropy so it would be unnecessary to implement unit tests for these. Unit tests

[24] https://jestjs.io/
[25] https://docs.pytest.org/en/stable/

could also be added for the backend endpoints, making requests to each endpoint and verifying that they produce the expected response.

9.5. Developer-facing documentation *Jianing Li*

There is a lack of developer facing documentation in our codebase, which is a potential enhancement that would improve the maintainability of our app. Currently, the README file contains a brief description of the project, the technologies used, and instructions for setting up a development environment. However, adding more documentation on the project would help developers better understand the codebase when modifying or extending it.

For example, having documentation on the architecture of the project detailing the high-level structure and key components of the application could be added, which would provide developers with the necessary context to understand how these components interact together. Additionally, documentation on code structure, including the files that important logic is located and where configuration is stored, would allow future developers to be familiarised with the codebase more quickly. This documentation could initially be in the form of Markdown files in a `docs/` folder, and subsequently be hosted on a static site if required.

10. Conclusion

10.1. Project aims and objectives

Jennifer Butcher

OPIS was developed around five core objectives: to be course-specific, intuitive, scientifically accurate, reliable, and modifiable. These objectives have guided the development process and provided a basis for assessing the success of our final application. Overall, we believe that OPIS successfully achieves all five objectives, although some have been addressed more fully than others.

OPIS is tailored to the ASTRO 100 course and provides all the functionality required to complete the laboratory worksheet. User testing demonstrated that the application is intuitive to use, while performance testing on lab computers demonstrated that it operates reliably in the intended teaching environment. Furthermore, OPIS runs locally on individual computers, allowing multiple students to use the application concurrently without relying on shared server resources. The use of REBOUND for orbital simulations, alongside data from reputable sources, supports the scientific accuracy of the simulations.

The objective of modifiability was less fully realised. Planetary systems are defined through individual JSON configuration files, with a provided template making it easier to introduce new systems or modify existing ones. This allows the ASTRO 100 teaching team to adapt the application's content as the course evolves. However, limitations with documentation and automated testing mean further work is needed to make OPIS truly easy to adapt. Similarly, although OPIS incorporates accessibility features, limitations in keyboard navigation and screen-size responsiveness mean that this remains an area for improvement.

10.2. Key findings

Jennifer Butcher

The development of OPIS highlighted several key challenges for our team, which provided valuable lessons and considerations for future development projects.

- **Obtaining reliable astrophysical data**. Developing scientifically-grounded simulations required sufficiently detailed and reliable astrophysical data, which proved challenging to obtain. This difficulty was compounded by our initially limited domain knowledge, which made it harder to identify and evaluate appropriate sources. Where data was ambiguous, we had to weigh the benefits of its inclusion against the risk of introducing unsupported assumptions, and consider how much uncertainty we were willing to accept in our models. This highlighted the importance of critically evaluating data sources and recognising the limitations of the information available when developing scientifically-grounded applications.
- **Integrating external libraries**. Incorporating external libraries such as Luxon, Flatpickr, Floating UI, Three.js, and Meshline were beneficial as they provided key functionality to OPIS that would have otherwise required significant developmental effort. However, integrating these libraries often required significant adaptation and troubleshooting, much more than we anticipated. This demonstrated that library selection involves more than just identifying a tool with the desired functionality; the additional time and effort that would be required to integrate it is an equally important consideration.
- **Balancing scientific accuracy and visual design**. Presenting planetary systems in a way that was scientifically informative, but still easily accessible to a broad cohort of students, required compromises. For example, orbital distances and planetary sizes differ by several orders of magnitude, meaning that representing both to the same scale would make some objects difficult to distinguish or explore. We therefore had to make deliberate choices about how these features were visually represented, prioritising the clarity of orbital motion and the ability to interact with individual objects over strictly proportional scaling. This highlighted how scientific accuracy alone doesn't guarantee an effective educational tool; we had to make judgements between simplifications which improved usability, and those that risked misrepresenting the underlying science.
- **Managing project scope and development time**. Working in an unfamiliar domain, and with new technologies, meant that our understanding of appropriate approaches for implementation evolved throughout the project. As we gained new knowledge, some of our initial implementations had to

be revisited or reworked, which extended development time. Additionally, over development we also identified additional features and refinements that we wanted to incorporate, expanding the project scope. From this, we learned the importance of allowing room for unexpected challenges and changes in direction when estimating development time and defining project scope.

10.3. Major outcomes

Jennifer Butcher

OPIS provides interactive 3D simulations for all the planetary systems covered in the ASTRO 100 laboratory (the Solar System, Kepler-16, and TRAPPIST-1). OPIS accurately simulates the positions of objects in these systems using the Python library REBOUND to model orbital dynamics. By running locally, OPIS is able to be used without an internet connection or reliance on shared server resources. Although developed specifically for ASTRO 100, the application codebase is published on GitHub, making it accessible to other individuals and organisations who may find it useful for teaching or exploring planetary systems.

Overall, OPIS delivers a relevant and reliable tool that supports the objectives of the ASTRO 100 laboratory. Though there is space for further refinement in areas such as documentation, testing, and accessibility, the application succeeds in fulfilling its intended purpose, while providing a foundation that the teaching team can adapt and extend as the course evolves.

11. References

[1] E. C. Prima, A. R. Putri, and N. Rustaman, 'Learning solar system using PhET simulation to improve students' understanding and motivation', *Journal of Science Learning*, vol. 1, no. 2, p. 60, Mar. 2018, doi: 10.17509/jsl.v1i2.10239.

[2] O. Chernikova, N. Heitzmann, M. Stadler, D. Holzberger, T. Seidel, and F. Fischer, 'Simulation-Based Learning in Higher Education: A Meta-Analysis', *Review of Educational Research*, vol. 90, no. 4, pp. 499–541, Aug. 2020, doi: 10.3102/0034654320933544.

[3] '*Orbit Viewer*'. Accessed: Aug. 14, 2026. [Online]. Available: https://ssd.jpl.nasa.gov/tools/orbit_viewer.html

[4] '*Eyes on Exoplanets*'. Accessed: Aug. 14, 2026. [Online]. Available: https://eyes.nasa.gov/apps/exo/#/

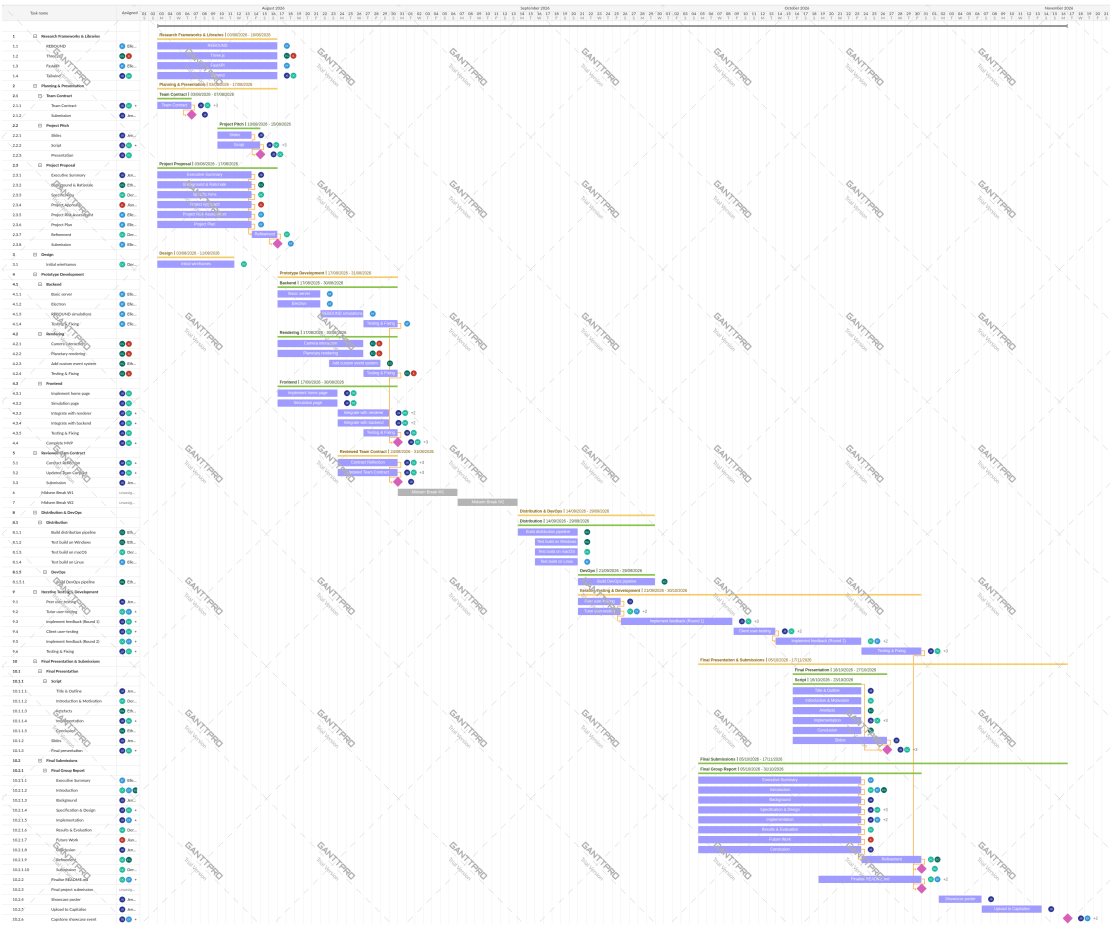
[5] X. Qiu and N. Qiu, 'Cognitive load and pedagogical tension in multi-platform online learning: Evidence from Chinese higher education', *PLOS One*, vol. 21, no. 4, p. e0347566, Apr. 2026, doi: 10.1371/journal.pone.0347566.

[6] 'REBOUND'. Accessed: Aug. 14, 2026. [Online]. Available: https://rebound.hanno-rein.de/

12. Appendices

12.1. Gantt chart

The revised Gantt chart for the project:



- Link to the revised Gantt chart[26]
- Link to the initial Gantt chart[27]

12.2. Final build

The final builds for OPIS can be found on the GitHub releases page[28] .

[26] https://app.ganttpro.com/shared/gantt/36610a744655d7384071eaff749d81c9ab61ad20036084398a4b43e166073e2f/3051575

[27] https://app.ganttpro.com/shared/gantt/3e42e667144e5436ef423275a9d8850e96a1af58b0ed9370f127b003e5dcf76e/3051581

[28] https://github.com/uoa-compsci399-s2-2026-anna/team-9-project/releases

12.3. Demonstration video

- Link to demonstration video[29]

12.4. Document links

- Link to Typst document[30] (final report formatting)
- Link to Google Docs document[31] (report draft)

[29] https://www.youtube.com/watch?v=4xJDFYLcwu8
[30] https://typst.app/project/rfb79TAkaQyVZ0BE6JymaN
[31] https://docs.google.com/document/d/1×7iHPKoddkZMEIYpJInXzYVzYgZugrVnj_GgyuzsE_M/edit?usp=sharing

12.5. Figures

Figure 1 NASA's *Eyes on Exoplanets* interface upon first launch. . . . 11
Figure 2 The home screen of OPIS. Students can select which system they wish to simulate, or using the controls in the top-right, display the help walkthrough, adjust OPIS' appearance, open the settings menu, and view OPIS in full screen. . . . 16
Figure 3 Students can use the play button in the bottom-centre of the page to play and pause the simulation. . . . 17
Figure 4 To show/hide objects such as Pluto, Halley's Comet, and ATLAS, students can first open the *Objects* side panel, and then select the object(s) they wish to show/hide. . . . 18
Figure 5 Users can rotate the simulation to show the inclinations of objects. . . . 18
Figure 6 Left: the 'Projection lines' view setting in the *View settings* side panel. Right: using the 'Projection lines' setting to see that part of Pluto's orbit comes closer to the Sun than Neptune's. . . . 20
Figure 7 Students can select the system picker dropdown to switch between systems. Once viewing Kepler-16, students can see that Kepler-16b is a planet orbiting the binary star system Kepler-16 (comprising Kepler-16A and Kepler-16B). . . . 21
Figure 8 Students can enable show the habitable zone overlay in the *View settings* side panel. . . . 22
Figure 9 Students can click the *Compare to Solar System*/*Hide Solar System* button to toggle the Solar System overlay. This overlays the orbits of the Solar System atop the current planetary system, which allows students to compare the orbits and scales of the two systems. It also allows students to see which (if any) planets of the Solar System are within the current system's habitable zone.
The habitable zone of TRAPPIST-1 is barely visible as a small green ring in the centre, while the orbit of Earth can be seen well outside of the habitable zone. . . . 23
Figure 10 Composite of OPIS' simulation UI in light mode (left) and dark mode (right). . . . 29
Figure 11 Light mode colour scheme. L-R: text, secondary, primary, accent colours. . . . 30
Figure 12 Dark mode colour scheme. L-R: text, secondary, primary, accent colours. . . . 30
Figure 13 Geist in regular weight. Geist is used for all body text within OPIS. . . . 30
Figure 14 OpenDyslexic in regular weight. OPIS offers OpenDyslexic as a font option for users with reading difficulties. . . . 30
Figure 15 Clockwise from top-left: settings menu; toolbar; tooltips (also showing keyboard shortcuts). . . . 31
Figure 16 Module-connector summary diagram of OPIS . 33
Figure 17 Backend startup state overview . . . 34
Figure 18 Renderer component and connector diagram . 35
Figure 19 Render loop . . . 35
Figure 20 State management component and connector diagram . . . 37
Figure 21 Event system component and connector diagram . . . 38
Figure 22 Diagram showing how the home page of OPIS is composed from templates, macros, and HTML. This is not an exhaustive representation of the true structure, but rather an indicative one which reflects the general hierarchy of macros and templates used to compose the home page. . 42
Figure 23 Left: the simulation playback controls. Right: Jinja macro `button_with_tooltip` used to display the play/pause button (right). . . . 43
Figure 24 Left: the rendered system picker dropdown. Right: Jinja template code for rendering each option in the system picker dropdown. . . . 44
Figure 25 OPIS' calendar in light and dark modes, implemented using Flatpickr. . . . 47
Figure 26 Diagram showing show event listeners are registered for buttons (`text_button` and `button_with_tooltip` macros). The button macros contain the data attribute `data-button-type`, which a JavaScript script queries and registers the events which are passed in as parameters to the macros. Then, each registered event is automatically fired when the button is clicked. . 50
Figure 27 A diagram of the implementation of interactivity for controls using the event system. An event is published as a result of a user interaction (see Figure 26 for more detail). These events are then subscribed to so that interactions can be handled, such as updating a tooltip as shown. . 51
Figure 28 Frame captured while the old simulation page is unloading. . . . 52
Figure 29 The help walkthrough of OPIS, showing a tooltip describing the toolbar. . . . 54
Figure 30 Floating UI automatically places tooltips so that they appear with the most space possible. Notice in the larger window, the tooltip is placed to the left of the target, while in the smaller window, the tooltip is placed beneath the target. . . . 55
Figure 31 Floating UI ensures that the tooltip's position is always updated to be next to its target. Here, the tooltip moves as the *Objects* panel is expanded. . . . 56
Figure 32 Simulation renderer flow chart . . . 57
Figure 33 Left: Pluto and Neptune without orbit line projections. From this perspective, it appears that Pluto's orbit is always further from the Sun than Neptune's. Right: a similar view with orbit line projections. From the projected lines, it is clear

that Pluto's orbit does come closer to the Sun than Neptune's. . . . 60
Figure 34 After rotating the camera, it is especially evident that Pluto's orbit comes closer to the Sun than Neptune's. . . . 61
Figure 35 The habitable zone overlay in OPIS. . . . 62
Figure 36 Tapered orbit lines in OPIS. . . . 62
Figure 37 The visual bug encountered due to the issue in `three.meshline`. Notice that the lateral sides of the orbits appear thicker than the vertical sides. This occurred when the screen aspect ratio was very uneven. . . . 64
Figure 38 The reference grid in OPIS. . . . 64
Figure 39 The object information panel for TRAPPIST-1d. . . . 66
Figure 40 Electron app composition overview . . . 70
Figure 41 Electron application lifespan phases diagram . 71
Figure 42 FastAPI launch state diagram . . . 73
Figure 43 Quick integration flow chart . . . 76
Figure 44 Backend `GET /systems` flow chart with priority events hitting the `DELETE /current_integration` endpoint . . . 79

12.6. Tables

Table 1 The tools and technologies utilised throughout the development of OPIS. . . . 12
Table 2 The JSON schema for the representation of a planetary system in the `config.json` file. . . . 24
Table 3 The JSON schema for the representation of an object (of a planetary system) in the `config.json` file. . . . 25
Table 4 The JSON schema for the representation of an object (of a planetary system)'s orbit in the `config.json` file. For more information, see REBOUND's orbital elements documentation[32] . . 26
Table 5 Detail of macros which are reused throughout OPIS. . . . 44
Table 6 Descriptions of the backend endpoints implemented in FastAPI. . . . 74
Table 7 Actions taken as a result of the first round of client feedback. . . . 84
Table 8 Actions taken as a result of the second round of client feedback. . . . 89
Table 9 Performance user requirements and whether they were successfully met in the final build of OPIS. . 90
Table 10 Functionality user requirements and whether they were successfully met in the final build of OPIS. . 91
Table 11 Deployment and packaging user requirements and whether they were successfully met in the final build of OPIS. . . . 92
Table 12 Accessibility user requirements and whether they were successfully met in the final build of OPIS. . . . 92
Table 13 A critical evaluation of how well OPIS achieved each of the aims and objectives described in Section 4.1 (Aims and objectives). . . . 93
Table 14 Signatures of authors . . . 109

12.7. Listings

Listing 1 A JSON snippet from the `config.json` file, indicative of how systems are represented in OPIS. . . . 28
Listing 2 Example of some Tailwind component classes used in OPIS. For example, `.clickable` adds hovering and active (actively being clicked) styling to buttons, which is responsive to dark and light mode. . . . 48
Listing 3 Custom Tailwind variant `short` used in OPIS. This variant is enabled based on a media query of the screen height and width. Then, a style is applied on an element when the variant is enabled: for example, when the `short` variant is active (i.e. the screen size is smaller than 1010×700 pixels), the `<span>` is hidden. . . . 48
Listing 4 Left: example of the call site of a button macro. Note the parameter `on_click_events`, where the event to be fired on click is passed. Right: the definition of the macro `button_with_tooltip`. Note the data attribute `data-button-type` on the parent `<div>`. . . . 49
Listing 5 In `simulation-playback-controls.js`, we subscribe to events published in response to UI interactions (e.g. buttons clicked, or inputs changed) to respond to these events. Here, when the simulation speed adjustment controls are interacted with, we ensure that the step forward/back buttons' tooltips are updated accordingly to reflect the new state. . . . 51

[32] https://rebound.hanno-rein.de/particles/orbitalelements/

12.8. Glossary

ecliptic plane: The plane of Earth's orbit around the Sun. 59, 59, 59

***IPC* – inter-process communication** 33, 34, 34, 38

non-stellar: A celestial object that is not a star. 97

perturbations: Influences of orbital objects on other objects. 77

pseudo-particle: A false orbital object used in osculating orbit calculations. 77, 77, 77